

PROJECT REPORT

on

EVENTFORCE MANAGEMENT SYSTEM – SALESFORCE IMPLEMENTATION

*A Salesforce CRM solution for event, client, vendor, venue and feedback management,
developed in a Salesforce Developer Edition org*

Submitted by

Preethi S	<i>Team Lead</i>
Sree Madhura A	<i>Team Member</i>
Safia Mehnaz S	<i>Team Member</i>
Saratha Swathi S	<i>Team Member</i>

A.V.C. College of Engineering

ABSTRACT

Event management involves coordinating clients, venues, vendors, reminders, cancellations and feedback. When these activities are maintained in registers or spreadsheets, information is fragmented, venue double bookings go undetected and managers have no single view of operations. This project, the **EventForce Management System – Salesforce Implementation**, addresses these problems by building an event management CRM in a Salesforce Developer Edition org.

The solution models Events, Clients, Vendors, Venues, Feedback and an EventVendor junction object as custom objects with lookup and master-detail relationships, a formula field, a validation rule and a lookup filter. A record-triggered Flow sends a client reminder three days before a confirmed event, and an approval process controls event cancellation requests. Apex classes and triggers update venue availability and prevent double booking, while batch and scheduled Apex mark past events as Completed. A Lightning App, a report and the EventForce Operations Dashboard provide visibility, and profiles, roles, a permission set, organisation-wide defaults and a sharing rule control access.

This report documents the requirement analysis, design, five implementation phases, test plan, results and limitations of the project. Supplied evidence confirms that the reminder Flow and the cancellation approval process are active and that the dashboard renders data from the Upcoming Events by Month report; other test cases are recorded as pending execution rather than passed. Actual production deployment was not performed; deployment preparation, maintenance and documentation approaches are described for future use. Limitations identified from the implementation guide are presented for verification together with suggested improvements.

Keywords: Salesforce, CRM, event management, Flow, Approval Process, Apex, Lightning App, dashboard.

TABLE OF CONTENTS

1. INTRODUCTION AND PROJECT OVERVIEW	1
1.1 Background	1
1.2 Project Overview	1
1.3 Purpose of the System	2
1.4 Development Environment and Basis of This Report	2
1.5 Organisation of the Report	3
2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE	4
2.1 Problem Statement	4
2.2 Objectives	4
2.3 Scope	5
2.4 Stakeholders	6
2.5 Alternatives Considered	6
3. REQUIREMENT ANALYSIS AND PROPOSED SYSTEM	7
3.1 Customer Journey	7
3.2 Functional Requirements	7
3.3 Non-Functional Requirements	8
3.4 Technology Stack	8
3.5 Problem–Solution Fit	9
3.6 Proposed System	10
3.7 Data Flow	10
4. SYSTEM ARCHITECTURE AND DATA MODEL	11
4.1 System Architecture	11
4.2 Data Model	13
4.3 Entity Relationship Diagram	14
4.4 Picklist Values, Formula and Validation Rule	16
4.5 Process Flow	17
5. PROJECT PLANNING AND METHODOLOGY	20
5.1 Planning Approach	20
5.2 Sprint Summary	20
5.3 Implementation Phases	21
6. PHASE 1: REQUIREMENT ANALYSIS AND PLANNING	22
6.1 Objective of the Phase	22
6.2 Functional Scope	22
6.3 Stakeholder Mapping	22
6.4 Execution Roadmap	23
6.5 Outcome of the Phase	23
7. PHASE 2: BACKEND DEVELOPMENT AND CONFIGURATIONS	24
7.1 Salesforce Developer Account (Milestone 1)	24

7.2 Custom Objects and Tabs (Milestones 2 and 3)	24
7.3 Fields and Relationships (Milestone 4)	25
7.4 Validation Rule (Milestone 5)	28
7.5 Approval Process for Event Cancellation (Milestone 6)	29
7.6 Flow: 3-Day Client Reminder (Milestone 7)	30
7.7 Apex Classes and Triggers (Milestones 8 and 9)	32
7.8 Batch and Scheduled Apex (Milestone 10)	33
8. PHASE 3: UI/UX DEVELOPMENT AND CUSTOMISATION	34
8.1 Lightning App (Milestone 11)	34
8.2 Reports (Milestone 12)	35
8.3 Dashboard (Milestone 13)	36
9. PHASE 4: DATA MIGRATION, TESTING AND SECURITY	37
9.1 Data Migration	37
9.2 Profiles	38
9.3 Roles and Users	39
9.4 Permission Set	40
9.5 Sharing Settings	41
10. PHASE 5: DEPLOYMENT, DOCUMENTATION AND MAINTENANCE	43
10.1 Deployment	43
10.2 Maintenance, Monitoring and Troubleshooting	44
10.3 Project Documentation	46
11. TESTING AND TEST CASES	47
11.1 Test Approach	47
11.2 Test Cases	47
11.3 Test Execution Summary	51
11.4 Test Evidence Placeholders	51
12. RESULTS AND DISCUSSION	55
12.1 Implementation Status	55
12.2 Findings from the Supplied Evidence	56
12.3 Discussion	56
13. LIMITATIONS AND FUTURE ENHANCEMENTS	58
13.1 Advantages and Disadvantages	58
13.2 Items Identified in the Implementation Guide	58
13.3 Additional Observations for Verification	60
13.4 Future Enhancements	60
14. CONCLUSION	62
REFERENCES	63
APPENDIX A: SOURCE CODE AND EXPRESSIONS	64
A.1 Event Budget Formula (Event__c)	64
A.2 Validation Rule Email_Valid_Address (Client__c)	64
A.3 VenueStatusHelper (Apex class)	64

A.4 EventTrigger13 (Apex trigger)	64
A.5 PreventDoubleBooking (Apex trigger)	65
A.6 BatchCompleteEvents (Batch Apex)	65
A.7 ScheduleCompleteEvents (Schedulable Apex)	65
APPENDIX B: EMAIL TEMPLATES	66
B.1 Event Cancellation Request Notification	66
B.2 Email to Client After Cancellation (not attached to an action – see L2)	66
B.3 Approval Notification to Event Owner	66
B.4 Rejection Notification to Event Owner (see L1)	66
B.5 Flow Text Templates	66
APPENDIX C: SCREENSHOT PLACEHOLDER REGISTER	68
APPENDIX D: IMPLEMENTATION EVIDENCE SUPPLIED BY THE TEAM	70

LIST OF FIGURES

Figure 4.1: Conceptual solution architecture of EventForce	12
Figure 4.2: Conceptual entity relationship diagram of EventForce	15
Figure 4.3: Conceptual event lifecycle process flow	18
Figure 4.4: Conceptual approval flow of Event_Cancellation_Process	19
Figure 7.1: Salesforce Setup home page of the Developer Edition org	24
Figure 7.2: Custom objects in Object Manager	25
Figure 7.3: Custom object tabs	25
Figure 7.4: Fields of the Event object	26
Figure 7.5: Fields of the Client object	26
Figure 7.6: Fields of the Vendor and Venue objects	26
Figure 7.7: Fields of the Feedback object	27
Figure 7.8: EventVendor junction object relationships	27
Figure 7.9: Event Budget formula field	27
Figure 7.10: Lookup filter on the Feedback Event field	28
Figure 7.11: Validation rule Email_Valid_Address	28
Figure 7.12: Classic email templates	29
Figure 7.13: Event_Cancellation_Process in Approval Processes	30
Figure 7.14: Approval process detail page with actions	30
Figure 7.15: Client Reminder flow in the Flows list	31
Figure 7.16: Flow canvas of the 3-day reminder Flow	31
Figure 7.17: Apex classes in Setup	32
Figure 7.18: Apex triggers in Setup	32
Figure 7.19: Apex source code in the Developer Console	33
Figure 7.20: Scheduled job Daily Event Completion	33
Figure 8.1: Event Planner Lightning App	34
Figure 8.2: App Manager entry and app settings	34
Figure 8.3: Report folder and sharing	35
Figure 8.4: Upcoming Events by Month report	36
Figure 8.5: EventForce Operations Dashboard	36
Figure 9.1: Data Import Wizard	37
Figure 9.2: Sample records in EventForce objects	38
Figure 9.3: Custom profiles	39

Figure 9.4: Object permissions of a custom profile	39
Figure 9.5: Role hierarchy	40
Figure 9.6: Users with roles and profiles	40
Figure 9.7: Feedback Manager permission set and assignment	41
Figure 9.8: Organisation-wide defaults for Event	41
Figure 9.9: Sharing rule Event_Sharing_For_Vendors	42
Figure 11.1: Evidence for TC-04: invalid e-mail rejected	52
Figure 11.2: Evidence for TC-09: double booking prevented	52
Figure 11.3: Evidence for TC-11: venue status set to Reserved	52
Figure 11.4: Evidence for TC-14: reminder e-mail	53
Figure 11.5: Evidence for TC-17 to TC-19: approval process execution	53
Figure 11.6: Evidence for TC-20: batch completion of past events	53
Figure 11.7: Evidence for TC-22 to TC-24: profile access	54

LIST OF TABLES

Table 1.1: Evidence categories used in this report	2
Table 2.1: Customer problem statements	4
Table 2.2: Stakeholders and their responsibilities	6
Table 2.3: Approaches considered for event management	6
Table 3.1: Customer journey stages	7
Table 3.2: Functional requirements and the components that address them	7
Table 3.3: Non-functional requirements	8
Table 3.4: Technology stack	8
Table 3.5: Problem–solution fit	9
Table 3.6: Summary of the proposed system	10
Table 4.1: Architecture components	13
Table 4.2: Custom objects and fields	13
Table 4.3: Object relationships	16
Table 4.4: Picklist values defined in the guide	16
Table 4.5: Event Budget formula (fixed estimate by event type)	16
Table 5.1: Product backlog and sprint allocation	20
Table 5.2: Story points per sprint (computed from the backlog)	20
Table 6.1: Execution roadmap	23
Table 7.1: Custom object creation settings	24
Table 7.2: Configuration of Event_Cancellation_Process (from the guide)	29
Table 7.3: Configuration of the 3-day reminder Flow (from the guide)	31
Table 8.1: Definition of the Upcoming Events by Month report	35
Table 9.1: Custom profiles and object permissions	38
Table 9.2: Sharing configuration	41
Table 10.1: EventForce metadata prepared for future migration	43
Table 10.2: Deployment options for future migration	44
Table 10.3: Monitoring of EventForce automation	45
Table 10.4: Troubleshooting guide	45
Table 11.1: Test status definitions	47
Table 11.2: Test cases: data model and validation	47
Table 11.3: Test cases: automation	48
Table 11.4: Test cases: security and access	50

Table 11.5: Test cases: reporting, application and migration	50
Table 11.6: Summary of planned and executed tests	51
Table 12.1: Implementation status by component	55
Table 12.2: Objectives versus implementation and evidence	56
Table 13.1: Guide-identified limitations and items requiring verification	59
Table 13.2: Additional observations requiring verification	60
Table C.1: Screenshot placeholders to be completed	68
Table D.1: Description of supplied evidence	70

1. INTRODUCTION AND PROJECT OVERVIEW

1.1 Background

Event management companies coordinate a large number of interdependent activities: capturing client requirements, booking venues, engaging vendors such as caterers and decorators, sending reminders, handling cancellations and collecting feedback after the event. When these activities are maintained in registers, spreadsheets or separate tools, information becomes fragmented, double bookings go unnoticed and managers lack a single view of operations.

Customer Relationship Management (CRM) platforms address this problem by keeping related business data in one place and automating routine processes. Salesforce is a cloud-based CRM platform that allows an organisation to model its own business data as custom objects and to automate processes using declarative tools (flows, approval processes, validation rules, formula fields) and programmatic tools (Apex classes, triggers, batch and scheduled jobs) [3], [4].

1.2 Project Overview

The **EventForce Management System** is a Salesforce-based CRM solution built to centralise and streamline event planning operations. It manages Events, Clients, Vendors, Venues and Feedback using a structured data model of six custom objects, including an EventVendor junction object that supports the many-to-many relationship between events and vendors [1].

The solution combines the following Salesforce capabilities, each of which is described in the later chapters of this report:

- custom objects, relationships, formula fields and validation rules for structured, accurate data;
- a record-triggered Flow that sends a client reminder three days before a confirmed event;
- an Approval Process (Event_Cancellation_Process) that controls event cancellation requests;
- Apex triggers and classes for venue availability updates and double-booking prevention, together with batch and scheduled Apex that mark past events as Completed;

- a Lightning App, a report and the EventForce Operations Dashboard for operational visibility;
- profiles, roles, permission sets, organisation-wide defaults and sharing rules for controlled access.

1.3 Purpose of the System

The purpose of EventForce is to provide one platform on which event teams can plan and monitor events from enquiry to feedback. In particular, the system is intended to:

- centralise client, event, venue, vendor and feedback information;
- streamline event booking and venue reservation;
- prevent double booking of venues through automated Apex logic;
- automate event reminders and cancellation approvals;
- improve coordination between clients, event coordinators, vendor managers and venue managers;
- provide reports and dashboards for operational visibility and decision-making.

1.4 Development Environment and Basis of This Report

All configuration, customisation and automation described in this report were developed in a **Salesforce Developer Edition org**. A Developer Edition org is a standalone environment for learning and development; it is not connected to a production org. **Actual production deployment was not performed** as part of this project (see Chapter 10).

This report is based on the team’s EventForce implementation guide [1], which records the intended design, the configuration steps and the source code of the solution, and on the implementation evidence that the team has supplied so far. To keep the report accurate, each statement about the state of the system is qualified using the evidence categories in Table 1.1. The report does not claim that a feature has been tested or has produced a particular result unless evidence for it has been supplied.

Table 1.1: Evidence categories used in this report

Category	Meaning
Documented in guide	The component or step is specified in the implementation guide (design, steps or source code). It does not by itself prove that the component was executed or tested.
Evidence supplied	The team has supplied a Salesforce screenshot that confirms the component exists or is in a stated state (for example, active).

Category	Meaning
Screenshot pending	A labelled placeholder is provided in the report; the team will insert the Salesforce screenshot.
Pending execution	A planned test case that has not yet been executed, or whose execution is not confirmed by evidence. No result is recorded.
Verified (evidence supplied)	A test or check whose outcome is confirmed by the supplied evidence.

1.5 Organisation of the Report

Chapter 2 states the problem, objectives and scope. Chapter 3 presents the requirement analysis and the proposed system. Chapter 4 describes the system architecture and data model. Chapter 5 outlines the planning approach. Chapters 6 to 10 describe the five implementation phases. Chapter 11 presents the test plan and test cases, Chapter 12 the results and discussion, Chapter 13 the limitations and future enhancements and Chapter 14 the conclusion. References and appendices follow.

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

2.1 Problem Statement

Event operations teams currently depend on manual registers or spreadsheets. Client details, event schedules, venue reservations, vendor assignments, reminders, cancellations and feedback are handled separately, which makes the complete event lifecycle difficult to coordinate. Table 2.1 summarises the problems reported by each stakeholder persona in the project documentation [1].

Table 2.1: Customer problem statements

ID	Persona	Wants to	Barrier / Cause	Feels
PS-1	Event Operations Manager	Manage event planning operations efficiently	Client, event, venue and vendor information is hard to manage centrally	Frustrated, concerned
PS-2	Event Coordinator	Create and manage event bookings efficiently	Event, client and venue details need many manual activities; no central platform for the event lifecycle	Overwhelmed, frustrated
PS-3	Venue Manager	Manage venue availability accurately	The same venue may be assigned to several events; availability tracking is needed	Concerned, uncertain
PS-4	Vendor Manager	Coordinate vendors for different events	Vendor details and event assignments are managed separately; events need several vendors	Overwhelmed, concerned
PS-5	Event Coordinator	Communicate event information and reminders on time	Reminders are handled manually; clients must be notified before events	Stressed, concerned
PS-6	Event Administrator	Manage cancellation requests systematically	Cancellations need approval and status updates under a controlled process	Frustrated, uncertain
PS-7	Event Coordinator	Collect and manage client feedback	Feedback must be maintained together with event and client details	Concerned about satisfaction
PS-8	Event Operations Manager	Monitor event operations and performance	Operational data is not visible in one place; needs reports and dashboards	Concerned, less confident in decisions

2.2 Objectives

The primary objective is to build a Salesforce CRM platform that digitises and automates event management operations. The specific objectives are:

- to model Events, Clients, Vendors, Venues, Feedback and Event–Vendor assignments in a structured Salesforce data model;
- to enforce data accuracy through validation rules and formula fields;
- to prevent venue double bookings and keep venue availability up to date using Apex logic;
- to automate client reminders and event status updates;
- to provide a controlled approval process for event cancellations;
- to provide role-based access to data through profiles, roles, permission sets and sharing settings;
- to provide a Lightning App, reports and a dashboard for real-time operational visibility;
- to document the implementation, deployment preparation and maintenance approach for future use.

The project brief also states business-value targets of reducing manual processes by about 60% and improving booking efficiency by about 40% [1]. These are design targets stated in the brief; they have not been measured in this project and are not reported as results.

2.3 Scope

2.3.1 In scope

- six custom objects with tabs, fields, relationships, a formula field, a validation rule and a lookup filter;
- one record-triggered Flow, one approval process with email templates, Apex classes, triggers, batch and scheduled Apex;
- a Lightning App named Event Planner, one report (Upcoming Events by Month) and one dashboard (EventForce Operations Dashboard);
- four custom profiles, four roles, one permission set, organisation-wide default for Event and one sharing rule;
- data loading through the Data Import Wizard, test planning and a deployment-preparation and maintenance approach.

2.3.2 Out of scope

- production deployment or migration to a live Salesforce org;
- integration with external systems, payment handling and mobile application development;

- performance or load testing and Apex unit-test development (no test classes are provided in the guide).

2.4 Stakeholders

Table 2.2: Stakeholders and their responsibilities

Stakeholder	Responsibility in EventForce
Clients	Request event services, provide feedback and view event updates.
Event Coordinators	Manage event scheduling, client communication and approval workflows.
Vendors / Vendor Managers	Handle assigned event services such as catering, decor and logistics.
Venue Managers	Manage venue availability and reservations.
Event Administrators	Oversee system operations, security and data monitoring.

2.5 Alternatives Considered

Four approaches were considered during ideation [1]. Table 2.3 compares them.

Table 2.3: Approaches considered for event management

Approach	Description	Key limitation
Manual registers	Maintain event, client, venue and vendor details manually.	Time-consuming and difficult to maintain.
Spreadsheet tracking	Track bookings and event data in spreadsheets.	Limited automation and real-time coordination.
Standalone portal	Develop a dedicated event-management portal.	Higher development and maintenance effort.
Salesforce CRM (selected)	Centralise event operations on the Salesforce platform.	Requires proper configuration and implementation.

Salesforce CRM was selected for its automation capability (reminders, approvals, status updates), scalability and real-time visibility through reports and dashboards.

3. REQUIREMENT ANALYSIS AND PROPOSED SYSTEM

3.1 Customer Journey

The customer journey from enquiry to closure defines the processes that the system must support. Table 3.1 lists the eight stages.

Table 3.1: Customer journey stages

Stage	Journey stage	What happens
1	Event enquiry and client registration	Client requirements and contact details are captured in Salesforce.
2	Event planning and booking	Event details such as type, date, budget and status are created and managed.
3	Venue selection and reservation	Suitable venues are identified, availability is checked and reservations are managed while preventing double bookings.
4	Vendor coordination	Vendors and required services are assigned to events and their status is coordinated.
5	Event confirmation and communication	Confirmed events trigger automated communication, including a reminder to the client before the event.
6	Event execution and monitoring	Progress is monitored through Salesforce records, reports and dashboards.
7	Cancellation and approval	Cancellation requests are submitted through the approval process with status updates and notifications.
8	Feedback and closure	Client feedback is captured after the event and completed events are updated for tracking and analysis.

3.2 Functional Requirements

Table 3.2: Functional requirements and the components that address them

FR	Requirement	Addressed by (per implementation guide)
FR-1	Event management	Event__c object, tab, fields (type, date, status, budget), lookups
FR-2	Client management	Client__c object, tab, fields, email validation rule
FR-3	Venue management	Venue__c object, tab, fields, availability status
FR-4	Vendor management	Vendor__c object, tab, fields, service type, status
FR-5	Event–vendor assignment	EventVendor__c junction object (two master-detail relationships)
FR-6	Venue availability and double-booking prevention	VenueStatusHelper, EventTrigger13, PreventDoubleBooking

FR	Requirement	Addressed by (per implementation guide)
FR-7	Automated event reminders	Record-triggered Flow “Client Reminder - 3 Days Before”
FR-8	Event cancellation approval	Event_Cancellation_Process approval process and email templates
FR-9	Feedback management	Feedback__c object with Event and Client lookups, lookup filter
FR-10	Reports and dashboards	Upcoming Events by Month report; EventForce Operations Dashboard
FR-11	Event status automation	BatchCompleteEvents and ScheduleCompleteEvents (Apex)
FR-12	Data migration	Salesforce Data Import Wizard

3.3 Non-Functional Requirements

Table 3.3: Non-functional requirements

Attribute	Description	How it is addressed
Usability	Easy event management and navigation	Lightning App with tabs for all objects, reports and dashboards
Security	Controlled access to data	Profiles, roles, permission set, OWD and sharing rule
Reliability	Accurate automation, approvals and venue validation	Flow, approval process, Apex trigger validation
Performance	Efficient processing of event data	Bulk-oriented batch Apex (200 records per batch)
Availability	Cloud access for authorised users	Salesforce cloud platform
Scalability	Growth in events, clients, vendors, venues and users	Platform-based data model and automation
Maintainability	Ability to monitor and update automation	Flows, Apex, validation rules and security can be monitored and changed in Setup
Data integrity	Accurate records	Validation rule, pre-validated Data Import Wizard loads

3.4 Technology Stack

Table 3.4: Technology stack

Layer	Technology
Platform	Salesforce CRM – Developer Edition
User interface	Salesforce Lightning App

Layer	Technology
Database	Salesforce custom objects
Business logic	Flows, formula fields, validation rules
Approval	Salesforce Approval Process
Automation	Record-triggered Flow, Apex triggers
Processing	Batch Apex and Scheduled Apex
Security	Profiles, roles, permission sets, sharing rules, OWD
Reporting	Reports and dashboards

3.5 Problem–Solution Fit

Table 3.5: Problem–solution fit

Problem	EventForce solution	Benefit
Client, event, venue and vendor data is hard to manage centrally	Custom objects Event, Client, Venue, Vendor, Feedback, EventVendor	Centralised, structured data
Event booking needs many manual activities	Flows and automated processes	Less manual effort
Risk of venue double booking	Apex trigger and VenueStatusHelper	Venue availability validated
Vendor coordination is difficult to track	EventVendor junction object and Vendor records	Event–vendor assignment supported
Clients may not receive timely reminders	3-day reminder Flow	Automated reminder for confirmed events
Cancellations need controlled processing	Event_Cancellation_Process	Structured approval and status updates
Feedback must be linked to events	Feedback object with Event and Client relationships	Organised feedback
Operational data is hard to monitor	Report and EventForce Operations Dashboard	Better visibility
Different users need different access	Profiles, roles, permission set, OWD, sharing rule	Controlled, secure access

3.6 Proposed System

Table 3.6: Summary of the proposed system

Parameter	Description
Problem	Manual and fragmented event management
Solution	Salesforce CRM (EventForce Management System)
Innovation	Automated reminders, venue validation and cancellation approval
Intended impact	Faster event coordination and improved operational visibility
Business model	Centralised event planning and management
Scalability	Supports growing numbers of events, clients, vendors, venues and users

3.7 Data Flow

Client and event details entered by the event team are validated and stored as Event, Client, Venue and Vendor records. The system checks venue availability, assigns venue and vendors, and confirms the event. A confirmed event triggers the three-day reminder; after the event date the event is marked Completed, feedback is collected and reports and dashboards summarise the data. Cancellation requests follow the approval process, and venue booking validation is performed by an Apex trigger. The complete flow is shown in Figure 4.3 (Chapter 4) and the cancellation path in Figure 4.4.

4. SYSTEM ARCHITECTURE AND DATA MODEL

4.1 System Architecture

EventForce follows a layered architecture on the Salesforce platform. Users access the Lightning App (presentation layer); automation and business logic (Flow, approval process, Apex, validation and formula rules) act on the data layer of custom objects; security settings control what each user can see and do; and the Data Import Wizard loads data into the objects. Figure 4.1 shows the conceptual architecture and Table 4.1 lists the components.

EventForce Management System – Conceptual Solution Architecture

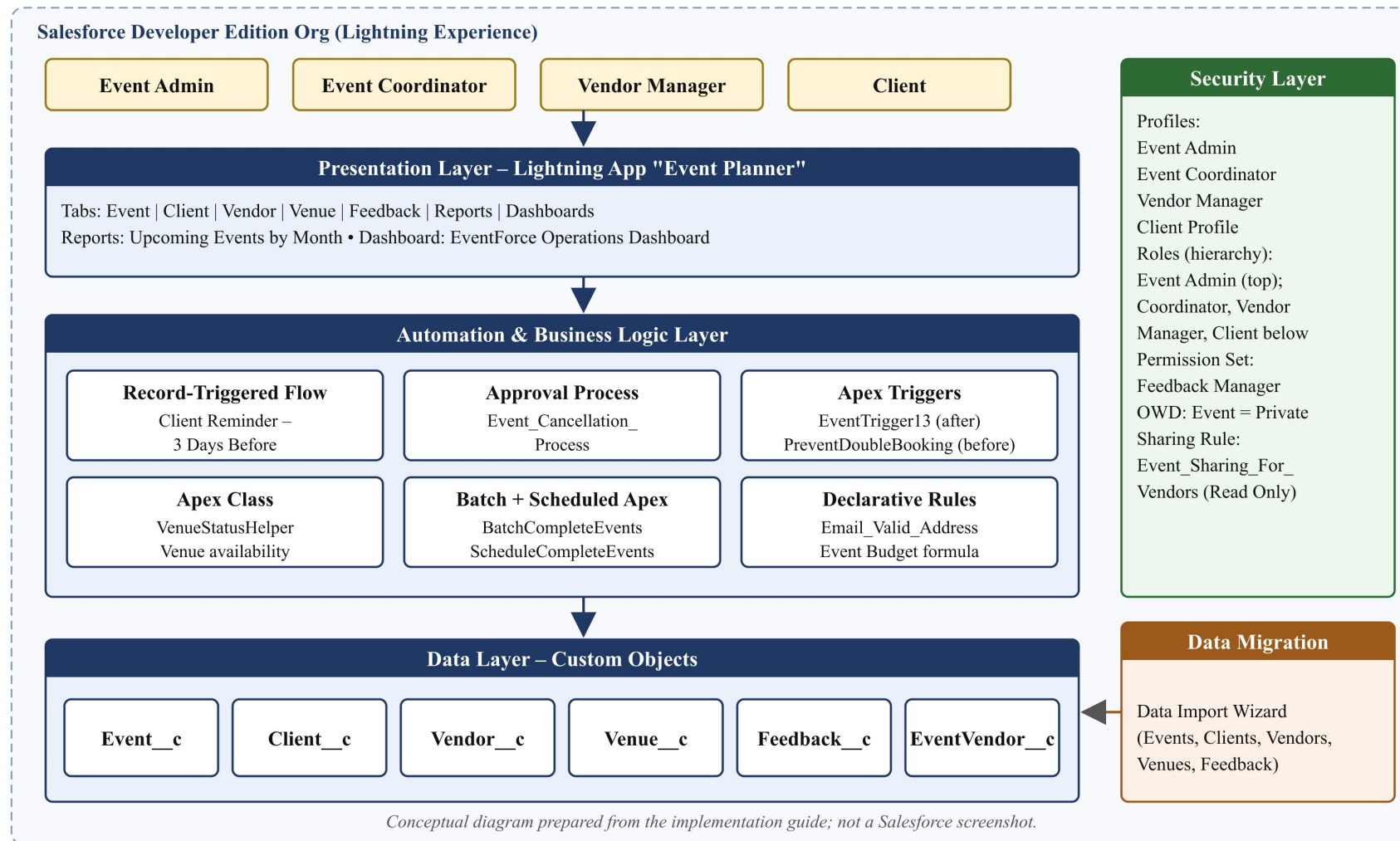


Figure 4.1: Conceptual solution architecture of EventForce

Table 4.1: Architecture components

Layer	Components
Presentation	Lightning App “Event Planner”, object tabs, report, dashboard
Data	Event__c, Client__c, Vendor__c, Venue__c, Feedback__c, EventVendor__c
Declarative logic	Validation rule Email_Valid_Address, Event Budget formula field, lookup filter on Feedback, Flow, Approval Process, email templates
Programmatic logic	VenueStatusHelper (class); EventTrigger13 and PreventDoubleBooking (triggers); BatchCompleteEvents and ScheduleCompleteEvents (batch and scheduled Apex)
Security	Profiles, roles, permission set, OWD, sharing rule
Data loading	Data Import Wizard

4.2 Data Model

The data model consists of five primary custom objects and one junction object. Table 4.2 lists the objects and their fields as specified in the implementation guide.

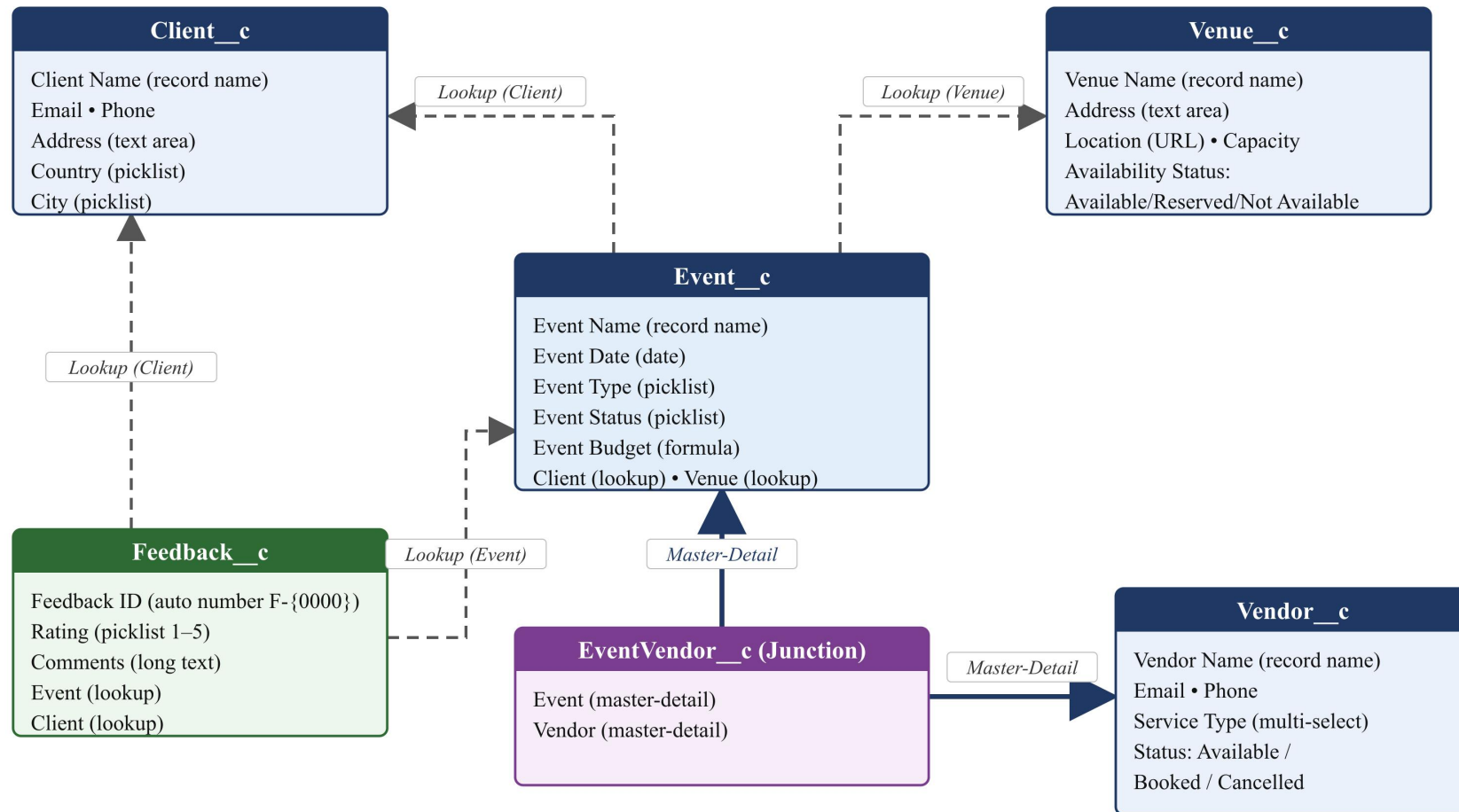
Table 4.2: Custom objects and fields

Object	API name	Purpose	Fields
Event	Event__c	Core record of an event	Event Name (text, record name); Event Date (date); Event Type (picklist); Event Status (picklist); Event Budget (formula, currency); Client (lookup); Venue (lookup)
Client	Client__c	Customer who books events	Client Name (text, record name); Email; Phone; Address (text area); Country (picklist); City (picklist)
Vendor	Vendor__c	Service provider for events	Vendor Name (text, record name); Email; Phone; Service Type (multi-select picklist); Status (picklist)
Venue	Venue__c	Location where events are held	Venue Name (text, record name); Address (text area); Location (URL); Capacity (number); Availability Status (picklist)
Feedback	Feedback__c	Client feedback about an event	Feedback ID (auto number, F-0000); Rating (picklist 1–5); Comments (long text); Event (lookup); Client (lookup)
Event Vendor	EventVendor__c	Junction object linking events and vendors	Event (master-detail); Vendor (master-detail)

4.3 Entity Relationship Diagram

Figure 4.2 shows the objects and relationships. Event is related to Client and Venue by lookup relationships. Feedback is related to Event and Client by lookup relationships. The many-to-many relationship between Event and Vendor is realised through EventVendor__c, which has two master-detail relationships.

EventForce Data Model – Conceptual Entity Relationship Diagram



Arrowheads point to the parent (referenced) object. Dashed = Lookup; solid = Master-Detail. Event ↔ Vendor many-to-many is realised through EventVendor__c.
 Conceptual diagram prepared from the implementation guide; not a Salesforce Schema Builder screenshot.

Figure 4.2: Conceptual entity relationship diagram of EventForce

Table 4.3: Object relationships

Relationship (child → parent)	Type
Event → Client	Lookup
Event → Venue	Lookup
Feedback → Event	Lookup (with lookup filter on the Feedback's Client)
Feedback → Client	Lookup
EventVendor → Event	Master-Detail
EventVendor → Vendor	Master-Detail

4.4 Picklist Values, Formula and Validation Rule

Table 4.4: Picklist values defined in the guide

Field	Values
Event → Event Type	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event → Event Status	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Client → Country	India, Philippines, Saudi Arabia, USA, Switzerland, Thailand, Maldives, Dubai, Sri Lanka, Bangkok, Singapore, Myanmar, France, Nepal
Client → City	Hyderabad, Mumbai, Delhi, Bangalore, Chennai, Alaminos, Angeles City, Palayan, Quezon City, San Juan, Riyadh, Jeddah, Tayma, Dammam, Medina, New York, Los Angeles, Chicago, Houston, Columbus, Geneva, Basel, Lausanne, Bern, Lucerne
Vendor → Service Type (multi-select)	Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor
Vendor → Status	Available, Booked, Cancelled
Venue → Availability Status	Available, Reserved, Not Available
Feedback → Rating	1, 2, 3, 4, 5

Table 4.5: Event Budget formula (fixed estimate by event type)

Event Type	Event Budget (currency value in formula)
Wedding	50,000
Corporate	30,000
Birthday	10,000
Anniversary	20,000

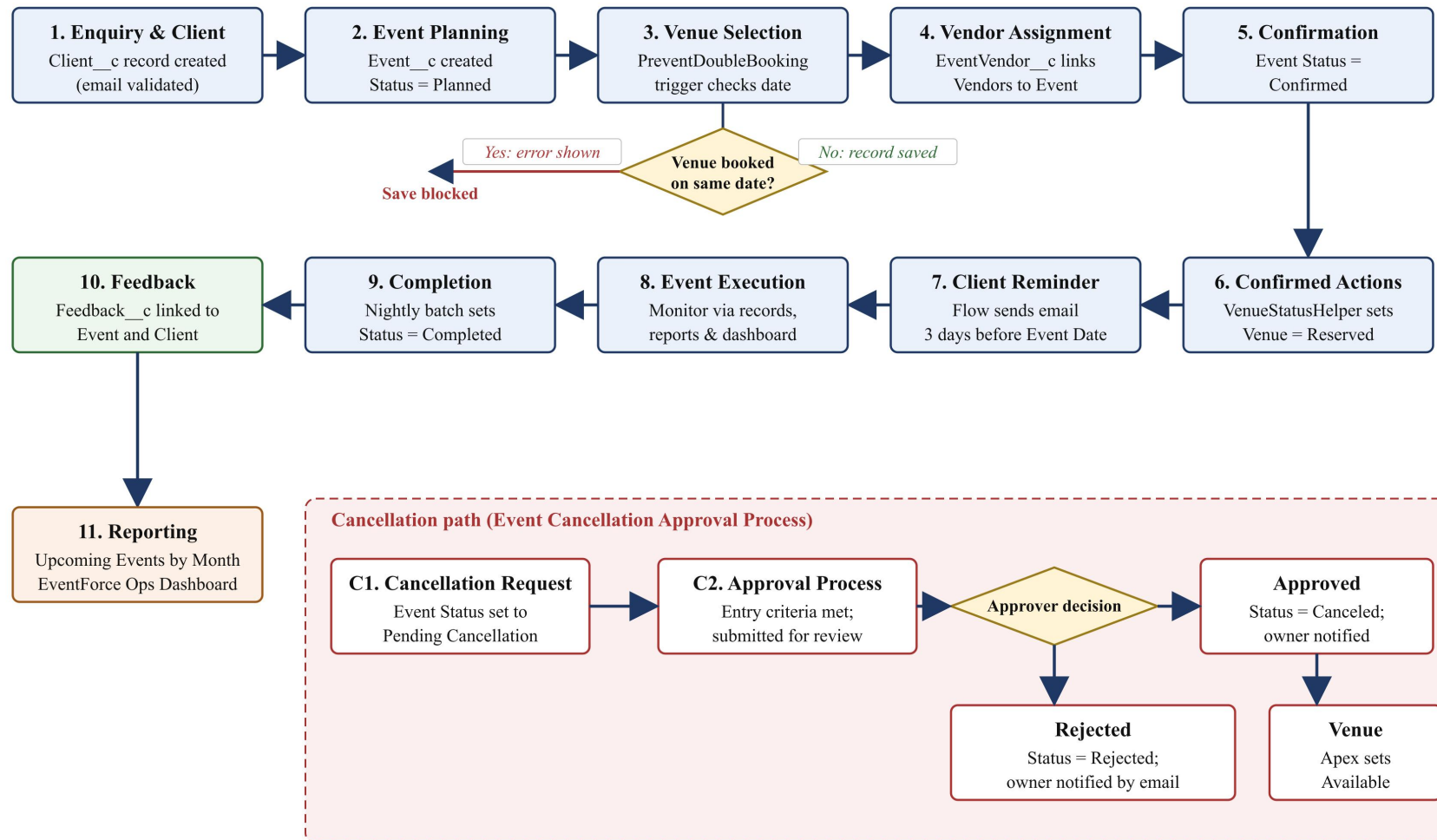
Event Type	Event Budget (currency value in formula)
Festival	60,000
Concert	40,000
Other	15,000
Any other / blank	0

The validation rule **Email_Valid_Address** on Client__c rejects an Email value that does not match a regular expression and shows the message “Please Enter Valid Email Address” on the Email field. The formula and rule expressions are listed in Appendix A.

4.5 Process Flow

Figure 4.3 shows the lifecycle of an event in EventForce, including the venue check, the confirmation actions, the nightly completion job and the cancellation path. Figure 4.4 details the cancellation approval process.

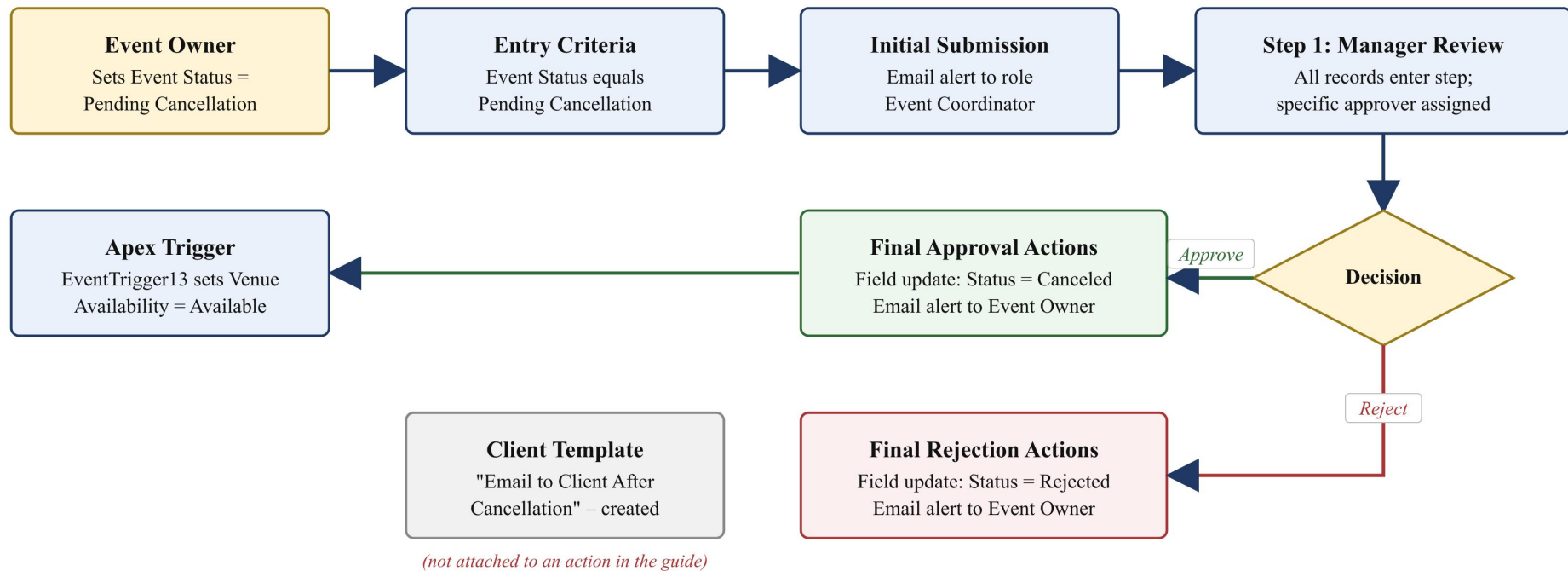
EventForce Event Lifecycle – Conceptual Process Flow



Conceptual diagram prepared from the implementation guide; not a Salesforce screenshot.

Figure 4.3: Conceptual event lifecycle process flow

Event_Cancellation_Process – Conceptual Approval Flow



Conceptual diagram prepared from the implementation guide; not a Salesforce screenshot.

Figure 4.4: Conceptual approval flow of Event_Cancellation_Process

5. PROJECT PLANNING AND METHODOLOGY

5.1 Planning Approach

The project was planned using an Agile approach: functional requirements (epics) were broken into user stories, each given story points, and grouped into four sprints. Table 5.1 presents the product backlog from the project documentation [1].

Table 5.1: Product backlog and sprint allocation

Sprint	Epic	ID	User story	Pts	Priority
Sprint-1	Event Management	US-01	As an Event Coordinator, I want to create events with type, date and status so that event information is standardised.	3	High
Sprint-1	Client & Venue Management	US-02	As an Event Coordinator, I want to associate a client and venue with an event so that coordination is centralised.	5	High
Sprint-2	Vendor Management	US-03	As a Vendor Manager, I want to assign multiple vendors to an event so that services can be coordinated.	5	High
Sprint-2	Notifications & Automation	US-04	As a Client, I want timely event reminders so that I know when my event is approaching.	3	High
Sprint-3	Approval Management	US-05	As an Event Coordinator, I want cancellation requests to follow an approval process so that cancellations are controlled.	5	High
Sprint-3	Venue Management & Apex	US-06	As a Venue Manager, I want double bookings prevented so that venue schedules remain accurate.	8	High
Sprint-4	Security & Access Control	US-07	As an Event Administrator, I want role-based access so that users see only appropriate information.	5	High
Sprint-4	Reports & Dashboards	US-08	As Management, I want reports and dashboards so that event operations can be monitored.	5	Medium

5.2 Sprint Summary

Table 5.2: Story points per sprint (computed from the backlog)

Sprint	User stories	Story points
Sprint-1	US-01, US-02	8
Sprint-2	US-03, US-04	8
Sprint-3	US-05, US-06	13
Sprint-4	US-07, US-08	10

Sprint	User stories	Story points
Total	8 stories	39

The sprint tracker in the original template lists 20 story points per sprint, with dates from the year 2022 and a completed-points entry for Sprint-1 only. Those figures do not agree with the backlog above and are template sample values, not project data. The actual sprint start and end dates, velocity and burndown data were not supplied and are therefore **not reported**. Team members' story assignments are likewise not stated, because they were not supplied.

5.3 Implementation Phases

The implementation was organised into five phases, which are described in Chapters 6 to 10:

1. **Phase 1** – Requirement Analysis and Planning;
2. **Phase 2** – Backend Development and Configurations;
3. **Phase 3** – UI/UX Development and Customisation;
4. **Phase 4** – Data Migration, Testing and Security;
5. **Phase 5** – Deployment, Documentation and Maintenance.

6. PHASE 1: REQUIREMENT ANALYSIS AND PLANNING

6.1 Objective of the Phase

The objective of Phase 1 was to define what the EventForce Management System must do before any configuration began: to help event planners manage client bookings, vendor coordination, venue reservations and feedback within a single CRM platform, to automate event scheduling, cancellation approvals and client reminders, and to provide real-time visibility through reports and dashboards [1].

6.2 Functional Scope

- custom objects Event, Client, Vendor, Venue, Feedback and EventVendor (junction object);
- relationship modelling using lookup, master-detail and many-to-many (Event ↔ Vendor) relationships;
- automation using Flows for reminders, feedback notifications and approval updates;
- validation rules and formula fields for accurate data entry and event budget calculation;
- an approval process for event cancellation requests, with email alerts to coordinators and clients;
- profiles, roles and permission sets for user-level access and security;
- reports and dashboards for event summaries, vendor performance and upcoming event insights.

Of the automation listed above, the implementation guide contains configuration steps for the three-day client reminder Flow and the cancellation approval process; feedback-notification flows are named in the scope but no steps are documented for them (see Chapter 13).

6.3 Stakeholder Mapping

The stakeholders (clients, event coordinators, vendors, venue managers and event administrators) and their responsibilities are listed in Table 2.2.

6.4 Execution Roadmap

Table 6.1: Execution roadmap

Step	Activity	Description
1	Object and field setup	Create the Event, Client, Vendor, Venue, Feedback and EventVendor objects.
2	Automation flows	Implement a record-triggered Flow for event reminders.
3	Validation rules and formula fields	Enforce data accuracy and budget calculation.
4	Profiles, roles and permission sets	Configure user access and record visibility.
5	Approval process	Automate the cancellation request workflow with notifications.
6	Apex classes and triggers	Update venue availability and prevent double bookings.
7	Reports and dashboards	Create analytics for upcoming events and performance insights.

6.5 Outcome of the Phase

Phase 1 produced the requirement set summarised in Chapter 3, the data model in Chapter 4 and the backlog in Chapter 5. The remaining phases implement this roadmap. Because the order of implementation in the guide differs slightly from the roadmap (the approval process and Apex logic are configured in Phase 2 and security in Phase 4), the guide notes that profiles, roles and users must be created before the approval process is implemented.

7. PHASE 2: BACKEND DEVELOPMENT AND CONFIGURATIONS

Phase 2 built the functional engine of EventForce using Salesforce Flows, Apex logic and declarative configuration. The steps below are those recorded in the implementation guide [1]; screenshot placeholders mark where the team’s Salesforce evidence is to be inserted.

7.1 Salesforce Developer Account (Milestone 1)

A Salesforce Developer Edition org was created through the Developer Edition sign-up page [5] using the Developer role. The account was activated by verifying the sign-up email, setting a password and security question, after which the Setup home page opens. All later work was carried out in this org.

[INSERT SCREENSHOT HERE – Figure 7.1]

Capture: Setup home page showing the org name “Developer Edition” and the logged-in user (avoid including passwords or personal e-mail addresses).

Delete this box after inserting the image; keep the caption below.

Figure 7.1: Salesforce Setup home page of the Developer Edition org

7.2 Custom Objects and Tabs (Milestones 2 and 3)

Six custom objects were created from Setup → Object Manager → Create → Custom Object. Table 7.1 summarises the creation settings. For each of the five primary objects a custom tab was created through Setup → Tabs so that the objects can be opened from the Lightning App.

Table 7.1: Custom object creation settings

Object	Record name	Options selected in the guide
Event	Event Name (text)	Allow Reports, Allow Activities, Track Field History, Deployed, Allow Search, Notes and Attachments related list
Client	Client Name (text)	Allow Reports, Allow Search, Notes and Attachments related list

Object	Record name	Options selected in the guide
Vendor	Vendor Name (text)	Allow Reports, Allow Search, Notes and Attachments related list
Venue	Venue Name (text)	Allow Reports, Allow Search, Notes and Attachments related list
Feedback	Feedback (auto number, display format F-{0000}, starting number 0)	Allow Reports, Allow Search, Notes and Attachments related list
Event Vendor	Junction object (API name EventVendor__c)	Two master-detail relationships to Event and Vendor

[INSERT SCREENSHOT HERE – Figure 7.2]

Capture: Setup → Object Manager list filtered to the six EventForce objects (Event, Client, Vendor, Venue, Feedback, Event Vendor), showing API names ending in __c.

Delete this box after inserting the image; keep the caption below.

Figure 7.2: Custom objects in Object Manager

[INSERT SCREENSHOT HERE – Figure 7.3]

Capture: Setup → Tabs page, “Custom Object Tabs” section listing the Event, Client, Vendor, Venue and Feedback tabs.

Delete this box after inserting the image; keep the caption below.

Figure 7.3: Custom object tabs

7.3 Fields and Relationships (Milestone 4)

Custom fields were added to each object using Object Manager → Fields & Relationships. The data types, picklist values and the Event Budget formula are listed in Chapter 4

(Table 4.2, Table 4.4 and Table 4.5). Relationships were created as follows: lookups from Event to Venue and Client; master-detail relationships from EventVendor to Event and Vendor; lookups from Feedback to Event and Client.

[INSERT SCREENSHOT HERE – Figure 7.4]

Capture: Object Manager → Event → Fields & Relationships showing Event Type, Event Date, Event Status, Event Budget, Client and Venue.

Delete this box after inserting the image; keep the caption below.

Figure 7.4: Fields of the Event object

[INSERT SCREENSHOT HERE – Figure 7.5]

Capture: Object Manager → Client → Fields & Relationships showing Email, Phone, Address, Country and City.

Delete this box after inserting the image; keep the caption below.

Figure 7.5: Fields of the Client object

[INSERT SCREENSHOT HERE – Figure 7.6]

Capture: Fields & Relationships pages of Vendor (Service Type, Email, Phone, Status) and Venue (Address, Location, Capacity, Availability Status); two images may be placed side by side.

Delete this box after inserting the image; keep the caption below.

Figure 7.6: Fields of the Vendor and Venue objects

[INSERT SCREENSHOT HERE – Figure 7.7]

Capture: Object Manager → Feedback → Fields & Relationships showing Rating, Comments, Event and Client.

Delete this box after inserting the image; keep the caption below.

Figure 7.7: Fields of the Feedback object

[INSERT SCREENSHOT HERE – Figure 7.8]

Capture: Object Manager → Event Vendor → Fields & Relationships showing the master-detail fields Event and Vendor.

Delete this box after inserting the image; keep the caption below.

Figure 7.8: EventVendor junction object relationships

[INSERT SCREENSHOT HERE – Figure 7.9]

Capture: Event Budget field detail page or edit page showing the CASE formula and the Currency return type.

Delete this box after inserting the image; keep the caption below.

Figure 7.9: Event Budget formula field

7.3.1 Lookup filter on Feedback

To make feedback consistent, a lookup filter was configured on the Event lookup of Feedback__c. After a client is selected on a Feedback record, the Event lookup lists only events belonging to that client: the filter compares Event: Client with Feedback: Client and the “Enable this filter” option is selected.

[INSERT SCREENSHOT HERE – Figure 7.10]

Capture: Feedback → Event field → Edit → Lookup Filter section with the filter criteria and “Enable this filter” checked.

Delete this box after inserting the image; keep the caption below.

Figure 7.10: Lookup filter on the Feedback Event field

7.4 Validation Rule (Milestone 5)

The validation rule **Email_Valid_Address** was created on Client__c. Its error condition formula is NOT(REGEX(Email__c, ...)) with the error message “Please Enter Valid Email Address” displayed on the Email field (full expression in Appendix A).

[INSERT SCREENSHOT HERE – Figure 7.11]

Capture: Client → Validation Rules → Email_Valid_Address detail page showing Active, error condition formula and error message.

Delete this box after inserting the image; keep the caption below.

Figure 7.11: Validation rule Email_Valid_Address

7.5 Approval Process for Event Cancellation (Milestone 6)

A controlled approval process was designed for event cancellations. The guide records the following configuration; the supplied Setup screenshot confirms that **Event_Cancellation_Process** exists as an active approval process on the Event object (see Chapter 12).

7.5.1 Email templates

Four classic text email templates were created: Event Cancellation Request Notification, Email to Client After Cancellation, Approval Notification to Event Owner and Rejection Notification to Event Owner. Their text is given in Appendix B.

[INSERT SCREENSHOT HERE – Figure 7.12]

Capture: Setup → Classic Email Templates list showing the four EventForce templates.

Delete this box after inserting the image; keep the caption below.

Figure 7.12: Classic email templates

7.5.2 Approval process configuration

Table 7.2: Configuration of Event_Cancellation_Process (from the guide)

Setting	Value
Object and name	Event; Event_Cancellation_Process
Entry criteria	Event Status equals Pending Cancellation
Approver selection	Automatically assigned (specific user / manager of the submitter)
Assignment email template	Event Cancellation Request Notification
Initial submitters	Event Owner
Initial submission action	Email alert “Alert - Event Cancellation Request” to role Event Coordinator
Approval step	Step “Manager Review”; all records enter the step; assigned to a specific user
Final approval actions	Field update Event Status = Canceled; email alert “Alert - Cancellation Approved” to the Event Owner

Setting	Value
Final rejection actions	Email alert “Alert - Cancellation Rejected” to the Event Owner; field update Event Status = Rejected
Activation	Process activated from its detail page

[INSERT SCREENSHOT HERE – Figure 7.13]

Capture: Setup → Approval Processes → Manage Approval Processes For: Event, showing the active process.

Note: A screenshot of this page was supplied by the team (Approval Processes for Event). Crop to the browser content and insert it here.

Delete this box after inserting the image; keep the caption below.

Figure 7.13: Event_Cancellation_Process in Approval Processes

[INSERT SCREENSHOT HERE – Figure 7.14]

Capture: Detail page of Event_Cancellation_Process showing entry criteria, approval step, and initial, approval and rejection actions.

Delete this box after inserting the image; keep the caption below.

Figure 7.14: Approval process detail page with actions

7.6 Flow: 3-Day Client Reminder (Milestone 7)

A record-triggered Flow named **Client Reminder - 3 Days Before** sends a reminder to the client. It runs on the Event object when a record is created or updated with Event Status equal to Confirmed, and uses a scheduled path that fires three days before the Event Date. The Flow was optimised for “Actions and Related Records”.

Table 7.3: Configuration of the 3-day reminder Flow (from the guide)

Element	Setting
Flow type	Record-triggered flow on Event; trigger: record created or updated
Entry condition	Event Status equals Confirmed
Scheduled path	Path name 3_Days_Before; time source Event Date; offset 3 days before
Text templates	Subject: “Reminder: Your Event {{\$Record.Name}} is in 3 days”; Body: event name, date, venue and type, signed “EventForce Team”
Action	Send Email – Alert_Client_3Day_Reminder; recipient: client email; CC: event owner; subject and body from the text templates
Flow label / activation	Client Reminder - 3 Days Before; saved and activated

[INSERT SCREENSHOT HERE – Figure 7.15]

Capture: Setup → Flows list showing “Client Reminder - 3 Days Before” as active.

Note: A screenshot of this page was supplied by the team (Flows list showing "Client Reminder - 3 Days Before"). Crop to the relevant rows and insert it here.

Delete this box after inserting the image; keep the caption below.

Figure 7.15: Client Reminder flow in the Flows list

[INSERT SCREENSHOT HERE – Figure 7.16]

Capture: Flow Builder canvas showing the start element, scheduled path 3_Days_Before and the Send Email action.

Delete this box after inserting the image; keep the caption below.

Figure 7.16: Flow canvas of the 3-day reminder Flow

7.7 Apex Classes and Triggers (Milestones 8 and 9)

Two pieces of Apex logic implement the venue rules. The class **VenueStatusHelper** sets Venue Availability Status to Reserved when an event is Confirmed and to Available when an event is Canceled. The trigger **EventTrigger13** (after insert, after update on Event__c) calls the helper. The trigger **PreventDoubleBooking** (before insert, before update) queries existing events at the same venue and adds the error “This Venue is already booked on this date.” when another event has the same venue and date. The source code, as recorded in the guide, is listed in Appendix A.

[INSERT SCREENSHOT HERE – Figure 7.17]

Capture: Setup → Apex Classes list showing VenueStatusHelper, BatchCompleteEvents and ScheduleCompleteEvents.

Delete this box after inserting the image; keep the caption below.

Figure 7.17: Apex classes in Setup

[INSERT SCREENSHOT HERE – Figure 7.18]

Capture: Setup → Apex Triggers list showing EventTrigger13 and PreventDoubleBooking on Event__c (status Active).

Delete this box after inserting the image; keep the caption below.

Figure 7.18: Apex triggers in Setup

[INSERT SCREENSHOT HERE – Figure 7.19]

Capture: Developer Console editor showing VenueStatusHelper or PreventDoubleBooking source code.

Delete this box after inserting the image; keep the caption below.

Figure 7.19: Apex source code in the Developer Console

7.8 Batch and Scheduled Apex (Milestone 10)

To mark past events as Completed every night, the batch class **BatchCompleteEvents** selects events whose date is before today and whose status is not Completed, sets their status to Completed and updates them. The class **ScheduleCompleteEvents** runs the batch with a scope of 200 records. A job named **Daily Event Completion** was scheduled from Setup → Apex Classes → Schedule Apex with weekly frequency on all days at 08:00 PM.

[INSERT SCREENSHOT HERE – Figure 7.20]

Capture: Setup → Scheduled Jobs page showing Daily Event Completion with the schedulable class and next run time.

Delete this box after inserting the image; keep the caption below.

Figure 7.20: Scheduled job Daily Event Completion

8. PHASE 3: UI/UX DEVELOPMENT AND CUSTOMISATION

Phase 3 created a simple interface that brings all EventForce modules into one workspace and added the analytics used to monitor events.

8.1 Lightning App (Milestone 11)

A Lightning App named **Event Planner** was created through Setup → App Manager → New Lightning App. The default app options and utility items were retained. The navigation items selected were Event, Client, Vendor, Venue, Feedback, Reports and Dashboards, and the app was made available to the System Administrator profile. Users open it from the App Launcher.

[INSERT SCREENSHOT HERE – Figure 8.1]

Capture: Event Planner app open from the App Launcher showing the navigation bar with Event, Client, Vendor, Venue, Feedback, Reports and Dashboards.

Delete this box after inserting the image; keep the caption below.

Figure 8.1: Event Planner Lightning App

[INSERT SCREENSHOT HERE – Figure 8.2]

Capture: Setup → App Manager showing the Event Planner app and, optionally, its selected navigation items and assigned profiles.

Delete this box after inserting the image; keep the caption below.

Figure 8.2: App Manager entry and app settings

8.2 Reports (Milestone 12)

A report folder named “EventForce Report” was created and shared, with View access, with the roles Event Coordinator and Vendor Manager. The report **Upcoming Events by Month** was then created on the Events with Client & Venue report type. Table 8.1 summarises its definition.

Table 8.1: Definition of the Upcoming Events by Month report

Element	Setting in the guide
Columns	Event Name, Event Date, Event Type, Client Name, Venue Name, Event Budget
Filters	Show Me: All Events; Event Date $\geq$ TODAY (or next 30 days); Event Status $\neq$ Canceled
Grouping	Event Date grouped by calendar month
Summaries	Record count; sum of Event Budget
Chart	Column or line chart: Event Date (month) against record count or sum of budget
Folder	EventForce Report folder

The guide advises creating at least ten records in every object, filling every field, before building the report so that the output is meaningful.

[INSERT SCREENSHOT HERE – Figure 8.3]

Capture: Reports tab showing the EventForce Report folder and the sharing dialog with the Event Coordinator and Vendor Manager roles (View).

Delete this box after inserting the image; keep the caption below.

Figure 8.3: Report folder and sharing

[INSERT SCREENSHOT HERE – Figure 8.4]

Capture: Report run page showing the filters, month grouping, summary values and chart.

Delete this box after inserting the image; keep the caption below.

Figure 8.4: Upcoming Events by Month report

8.3 Dashboard (Milestone 13)

The **EventForce Operations Dashboard** was created from the Dashboards tab. Its widget uses the Upcoming Events by Month report with a donut chart, a maximum of six displayed values, the subtitle “Upcoming Events Overview” and the light widget theme.

The supplied dashboard screenshot shows the dashboard open in Salesforce Lightning with the widget titled “Upcoming Events by Month.”, a donut chart whose centre shows a record count of 1, a legend entry for the event date 10/5/2026 and the timestamp “As of Oct 3, 2026, 7:13 AM”. This confirms that the dashboard and its source report exist and render; it also shows that only one upcoming event record was included in the report at that time (see Chapter 12).

[INSERT SCREENSHOT HERE – Figure 8.5]

Capture: Dashboard page showing the donut chart widget. Ideally refresh the dashboard after loading the sample records so that the chart reflects the full data set.

Note: A screenshot of this dashboard was supplied by the team (dashboard view dated Oct 3, 2026). Crop to the dashboard area and insert it here.

Delete this box after inserting the image; keep the caption below.

Figure 8.5: EventForce Operations Dashboard

9. PHASE 4: DATA MIGRATION, TESTING AND SECURITY

Phase 4 prepares the system for use: validated data is loaded, users are created and role-based access is configured. Testing for this and the earlier phases is described in Chapter 11.

9.1 Data Migration

According to the implementation guide, records for Events, Clients, Vendors, Venues and Feedback were imported using the **Salesforce Data Import Wizard** after pre-validation checks to avoid duplicates and maintain consistency. The guide does not contain the import steps, mapping files or import results, and none have been supplied; the import is therefore recorded here as **documented in the guide, evidence not yet supplied**.

Because Event and Feedback records reference Client and Venue records, loading the parent objects first (Client, Venue and Vendor, then Event, then Feedback and EventVendor) is the order that satisfies the lookup and master-detail dependencies described in Chapter 4. Source files should be checked for duplicate clients, invalid e-mail addresses (which the validation rule would reject) and event dates that conflict with existing venue bookings (which the double-booking trigger would reject).

[INSERT SCREENSHOT HERE – Figure 9.1]

Capture: Setup → Data Import Wizard page, or the wizard's mapping/confirmation step, or the Bulk Data Load Jobs page showing the completed import.

Delete this box after inserting the image; keep the caption below.

Figure 9.1: Data Import Wizard

[INSERT SCREENSHOT HERE – Figure 9.2]

Capture: List views of Event, Client, Vendor, Venue and Feedback records (one image per object or a combined image).

Delete this box after inserting the image; keep the caption below.

Figure 9.2: Sample records in EventForce objects

9.2 Profiles

Four custom profiles were created. Event Admin is cloned from System Administrator and therefore inherits full access. The other three are cloned from Standard Platform User and have Event Planner as the default custom app, a session timeout of two hours of inactivity, passwords that never expire and a minimum password length of eight characters. Table 9.1 summarises their object permissions.

Table 9.1: Custom profiles and object permissions

Profile	Base profile	Event	Client	Vendor	Venue	Feedback
Event Admin	System Administrator	Full	Full	Full	Full	Full
Event Coordinator	Standard Platform User	Create, Read, Edit, Delete	Read, Edit	Read	Read	Create, Read
Vendor Manager	Standard Platform User	Read	Read	Create, Read, Edit, Delete	Read	Read
Client Profile	Standard Platform User	Read (own events, as intended)	Not specified	No access	No access	Create, Read

[INSERT SCREENSHOT HERE – Figure 9.3]

Capture: Setup → Profiles list showing Event Admin, Event Coordinator, Vendor Manager and Client Profile.

Delete this box after inserting the image; keep the caption below.

Figure 9.3: Custom profiles

[INSERT SCREENSHOT HERE – Figure 9.4]

Capture: Profile detail page (for example Event Coordinator) showing the Custom Object Permissions section.

Delete this box after inserting the image; keep the caption below.

Figure 9.4: Object permissions of a custom profile

9.3 Roles and Users

A role hierarchy was created with **Event Admin** below the CEO role and the roles **Event Coordinator**, **Vendor Manager** and **Client** below Event Admin. Users were then created for each role: the Event Admin user with the Salesforce licence and the Event Admin profile, and the other users with the Salesforce Platform licence and the matching profile. The user name shown in the guide is a sample value from the guide template; user names and e-mail addresses are not reproduced in this report.

[INSERT SCREENSHOT HERE – Figure 9.5]

Capture: Setup → Roles showing Event Admin with Event Coordinator, Vendor Manager and Client beneath it.

Delete this box after inserting the image; keep the caption below.

Figure 9.5: Role hierarchy

[INSERT SCREENSHOT HERE – Figure 9.6]

Capture: Setup → Users list (or user detail pages) showing the role, profile and licence of each EventForce user. Mask e-mail addresses if required.

Delete this box after inserting the image; keep the caption below.

Figure 9.6: Users with roles and profiles

9.4 Permission Set

A permission set named **Feedback Manager** was created with Read, Create, Edit and Delete permissions on the Feedback object and assigned to the Event Coordinator user, so that additional access is granted without changing the profile.

[INSERT SCREENSHOT HERE – Figure 9.7]

Capture: Permission Set → Feedback Manager → Object Settings (Feedback permissions) and Manage Assignments page.

Delete this box after inserting the image; keep the caption below.

Figure 9.7: Feedback Manager permission set and assignment

9.5 Sharing Settings

The organisation-wide default for the **Event** object was set to Private for both internal and external access, so that only the record owner and users above the owner in the role hierarchy can access an event by default. An owner-based sharing rule, **Event_Sharing_For_Vendors**, then shares events owned by members of the Event Coordinator role with members of the Vendor Manager role with Read Only access.

Table 9.2: Sharing configuration

Item	Setting
OWD – Event	Default internal access: Private; default external access: Private
Sharing rule	Event_Sharing_For_Vendors
Records shared	Events owned by members of role Event Coordinator
Shared with	Role Vendor Manager
Access level	Read Only

[INSERT SCREENSHOT HERE – Figure 9.8]

Capture: Setup → Sharing Settings → Organization-Wide Defaults showing Event = Private.

Delete this box after inserting the image; keep the caption below.

Figure 9.8: Organisation-wide defaults for Event

[INSERT SCREENSHOT HERE – Figure 9.9]

Capture: Sharing Settings → Event Sharing Rules showing the rule definition.

Delete this box after inserting the image; keep the caption below.

Figure 9.9: Sharing rule Event_Sharing_For_Vendors

The guide describes the Client profile as having read access to “their own events”, but it does not document a sharing mechanism that restricts client users to their own events; this is recorded as an item requiring verification in Chapter 13.

10. PHASE 5: DEPLOYMENT, DOCUMENTATION AND MAINTENANCE

10.1 Deployment

10.1.1 Development environment and deployment status

All configurations, customisations and automations of EventForce were developed in a Salesforce Developer Edition org. A Developer Edition org is a standalone environment and is not connected to a production instance. **Actual production deployment – the migration of the solution from this development org to a live Salesforce environment – was not performed.** No deployment was executed, and no deployment logs, change sets or source-control repository have been supplied; this report therefore does not describe any deployment (including a simulated one) as having been carried out.

The purpose of this phase is to describe how the solution is prepared for deployment once configuration, testing and security setup are complete, so that migration can be carried out in future.

10.1.2 Deployment preparation

Preparation consists of ensuring that all metadata is complete, consistently named and ready to be moved. Table 10.1 lists the EventForce metadata that would form the deployment package.

Table 10.1: EventForce metadata prepared for future migration

Metadata type	EventForce components	Preparation notes
Custom objects and fields	Event__c, Client__c, Vendor__c, Venue__c, Feedback__c, EventVendor__c; picklists; Event Budget formula; relationships	Deploy parent objects before children; include tabs and page layouts
Validation rules	Email_Valid_Address	Included with the Client object
Flows	Client Reminder - 3 Days Before	Deploy as active only after testing in the target org
Approval process and email templates	Event_Cancellation_Process; four email templates; email alerts; field updates	Templates and alerts must exist before the process
Apex	VenueStatusHelper, EventTrigger13, PreventDoubleBooking, BatchCompleteEvents, ScheduleCompleteEvents	Requires test classes (see below); scheduled job must be re-created in the target org
Security	Profiles, roles, permission set, sharing rule, OWD	Users and role assignments are configured per org

Metadata type	EventForce components	Preparation notes
UI and analytics	Event Planner app, tabs, report folder, report, dashboard	Reports and dashboards need folder access to be re-applied

Before deployment to production, Apex code needs unit tests that provide the code coverage Salesforce requires (at least 75% overall) [4]. The implementation guide provides **no Apex test classes**, so test classes covering the triggers, helper class, batch class and schedulable class must be written first.

10.1.3 Deployment options

Table 10.2: Deployment options for future migration

Option	Description	Suitability for EventForce
Sandbox environments	Copies of the org for build, integration testing and user acceptance testing before production.	Recommended staging area for all changes.
Change sets	Point-and-click packages of metadata moved between connected orgs (for example sandbox to production).	Suitable for a small, one-time deployment; requires connected orgs.
Salesforce DX	Source-driven development with metadata retrieved to a local project and deployed with the Salesforce CLI [6].	Suitable for version-controlled, repeatable deployments.
DevOps tools / CI-CD	Automated pipelines (for example with Git-based repositories) that validate and deploy metadata.	Suitable for team-based, ongoing releases.

A recommended sequence is: retrieve metadata into a version-controlled project; deploy to a sandbox; run test classes and the test cases of Chapter 11; correct the known issues of Chapter 13; obtain user acceptance; and finally validate and deploy to production during a planned release window.

10.2 Maintenance, Monitoring and Troubleshooting

10.2.1 Maintenance

Maintenance keeps event, client, vendor and feedback data accurate and the configuration current. Typical activities are: reviewing and de-duplicating records; updating picklist values (for example event types, service types, cities); keeping venue availability and vendor status current; reviewing user access when staff change; and revising validation rules, flows and Apex when business rules change.

10.2.2 Monitoring

Table 10.3: Monitoring of EventForce automation

Area	What to monitor	Where in Salesforce
Flows	Failed or paused flow interviews; whether the reminder Flow is active	Setup → Flows; Paused and Failed Flow Interviews
Apex jobs	Outcome of the nightly Daily Event Completion job and batch	Setup → Scheduled Jobs; Apex Jobs
Apex errors	Unhandled exceptions in triggers and classes	Debug Logs; Developer Console; email notifications of Apex errors
Approval processes	Pending, approved and rejected cancellation requests	Approval History related list; Items to Approve
Data quality	Duplicates, missing e-mail addresses, inconsistent statuses	Reports and list views
Configuration changes	Who changed which setting	Setup Audit Trail

10.2.3 Troubleshooting

Table 10.4: Troubleshooting guide

Symptom	Possible cause	Suggested action
User cannot see an object, tab or app	Missing profile or permission-set access; app not assigned to the profile	Check profile object permissions, tab settings and app assignment; review permission sets
User cannot see a particular event	OWD Private; user not owner or above owner in hierarchy; no sharing rule	Check role hierarchy, OWD and the Event_Sharing_For_Vendors rule; consider sharing options
Permission conflict between profile and permission set	Permission sets only add access; another setting restricts it	Compare effective permissions in the user's profile and assigned permission sets
"This Venue is already booked on this date." on save	Another event uses the same venue and date	Confirm the clash is genuine; change venue or date; review trigger logic for cancelled events (see Chapter 13)
Venue availability status incorrect	Event status not Confirmed/Canceled; trigger error	Review VenueStatusHelper logic and debug logs
Reminder email not sent	Flow inactive; event not Confirmed; client e-mail missing; scheduled path time already passed	Check Flow activation and failed interviews; verify data on the event and client
Record cannot be saved – e-mail error	Validation rule Email_Valid_Address rejects the e-mail format	Correct the e-mail or review the regular expression
Cancellation request stuck	Approver not assigned or unavailable	Check approver configuration and reassign the approval request

Symptom	Possible cause	Suggested action
Past events not marked Completed	Scheduled job not running or batch failed	Check Scheduled Jobs, Apex Jobs and debug logs

10.3 Project Documentation

Project documentation is the complete record of EventForce. This report serves that purpose and records the following:

- **Business overview and objectives** – Chapters 1 and 2;
- **Phase-wise implementation** – Chapters 6 to 10 (requirement analysis, backend development, UI, data migration and security, deployment and maintenance);
- **Data model and ER diagram** – Chapter 4;
- **Automation components** – flow, approval process, validation rules and Apex logic in Chapters 7 and Appendices A and B;
- **Security model** – profiles, roles, permission set and sharing rules in Chapter 9;
- **Testing** – test cases and their execution status in Chapter 11;
- **Screenshots** – labelled screenshot locations for objects, fields, flows, approval process, Apex, security, reports and dashboards, indexed in Appendix C.

Together these provide a blueprint for future developers and administrators. They support maintenance by showing how each requirement maps to a Salesforce component, support training by describing each configuration in order, and support knowledge transfer by recording the known limitations (Chapter 13) and the deployment preparation steps described above.

11. TESTING AND TEST CASES

11.1 Test Approach

Testing of a Salesforce implementation covers configuration (are the objects, fields and settings present), functional behaviour of automation (do the validation rule, Apex logic, Flow and approval process behave as designed), security (does each profile see and do only what is intended) and reporting. The implementation guide states that testing activities validated data accuracy, automation and user permissions [1], but it does not include a test log. This chapter therefore separates **planned tests** from **tests actually performed**.

The expected results below are derived from the design in the implementation guide. The last column records the actual result using the statuses defined in Table 11.1. A test is shown as verified only where the team's supplied evidence confirms it.

Table 11.1: Test status definitions

Status	Meaning
Verified (evidence supplied)	The outcome is confirmed by a Salesforce screenshot supplied by the team. The confirmed scope is stated in the actual-result column.
Pending execution	Planned test; execution not confirmed by supplied evidence. No actual result is recorded and none should be assumed.
Not verified	The test cannot currently be confirmed because the guide does not document the necessary mechanism or artefact.

11.2 Test Cases

11.2.1 Data model and validation

Table 11.2: Test cases: data model and validation

ID	Test scenario / input	Expected result	Status and actual result
TC-01	Open Object Manager and check the six custom objects (Event, Client, Vendor, Venue, Feedback, Event Vendor).	All six objects are present, deployed and searchable, with API names ending in __c.	Pending execution No result recorded.
TC-02	Create an Event record with name, date, type, status, client and venue.	Record saves; Client and Venue lookups resolve; Event Budget is populated by the formula.	Pending execution No result recorded.

ID	Test scenario / input	Expected result	Status and actual result
TC-03	Set Event Type to each value in turn.	Event Budget equals 50,000 (Wedding), 30,000 (Corporate), 10,000 (Birthday), 20,000 (Anniversary), 60,000 (Festival), 40,000 (Concert), 15,000 (Other).	Pending execution No result recorded.
TC-04	Save a Client with an invalid e-mail (for example “abc@”).	Save is blocked; message “Please Enter Valid Email Address” appears on the Email field.	Pending execution No result recorded.
TC-05	Save a Client with a valid e-mail (for example name@example.com).	Record saves without error.	Pending execution No result recorded.
TC-06	Create Feedback: select a Client, then open the Event lookup.	Only events of the selected client are listed (lookup filter).	Pending execution No result recorded.
TC-07	Create EventVendor records: two vendors for one event; one vendor for two events.	Records save; related lists on Event and Vendor show the assignments.	Pending execution No result recorded.
TC-08	Create a Feedback record.	Feedback ID is generated automatically in the F-{0000} format.	Pending execution No result recorded.

11.2.2 Automation (Apex, Flow, approval process, batch)

Table 11.3: Test cases: automation

ID	Test scenario / input	Expected result	Status and actual result
TC-09	Create two events with the same Venue and same Event Date.	Second save is blocked with “This Venue is already booked on this date.”	Pending execution No result recorded.
TC-10	Create two events at the same Venue on different dates.	Both records save.	Pending execution No result recorded.
TC-11	Save an Event with a Venue and Event Status = Confirmed.	Venue Availability Status becomes Reserved.	Pending execution No result recorded.
TC-12	Set a Confirmed event’s status to Canceled.	Venue Availability Status becomes Available.	Pending execution No result recorded.

ID	Test scenario / input	Expected result	Status and actual result
TC-13	Open Setup → Flows and locate the reminder Flow.	Flow “Client Reminder - 3 Days Before” exists and is active.	Verified (evidence supplied) Flows list screenshot shows the Flow with Active ticked (Unmanaged; last modified by Preethi S, 10/3/2026, 3:06 AM). Run-time behaviour not covered.
TC-14	Confirm an event with a client having an e-mail; wait for the scheduled path (3 days before Event Date).	Client receives the reminder; event owner is copied.	Pending execution No result recorded.
TC-15	Save an event with a status other than Confirmed.	No reminder is scheduled or sent.	Pending execution No result recorded.
TC-16	Open Setup → Approval Processes for Event.	Event_Cancellation_Process exists and is active.	Verified (evidence supplied) Approval Processes screenshot shows Event_Cancellation_Process under Active Approval Processes with process order 1; no inactive processes. Run-time behaviour not covered.
TC-17	Set Event Status = Pending Cancellation and submit.	Record enters the approval process; Event Coordinator role receives the alert.	Pending execution No result recorded.
TC-18	Approve the cancellation request.	Event Status becomes Canceled; owner receives approval e-mail; venue made Available.	Pending execution No result recorded.
TC-19	Reject the cancellation request.	Event Status becomes Rejected; owner receives the rejection e-mail. (Check the e-mail text against the status – see Chapter 13, L1.)	Pending execution No result recorded.
TC-20	Create an event dated in the past with status not Completed; run the batch.	Event Status becomes Completed.	Pending execution No result recorded.
TC-21	Open Setup → Scheduled Jobs.	Job “Daily Event Completion” is scheduled daily at 08:00 PM.	Pending execution No result recorded.

11.2.3 Security and access

Table 11.4: Test cases: security and access

ID	Test scenario / input	Expected result	Status and actual result
TC-22	Log in as an Event Coordinator user and test each object.	Create/read/edit/delete on Event; read/edit on Client; read on Vendor and Venue; create/read on Feedback.	Pending execution No result recorded.
TC-23	Log in as a Vendor Manager user and test each object.	Full access to Vendor; read-only on Event, Venue, Client and Feedback.	Pending execution No result recorded.
TC-24	Log in as a Client Profile user and test each object.	Read-only on Event; create/read on Feedback; no access to Vendor and Venue.	Pending execution No result recorded.
TC-25	As a Client Profile user, list events.	Only the client's own events are visible (intended behaviour in the guide).	Not verified No sharing mechanism for this is documented in the guide (see Chapter 13, L6). Not verified.
TC-26	As a user who is not the owner and not above the owner in the role hierarchy, open another user's event.	Event is not visible (OWD Private).	Pending execution No result recorded.
TC-27	As a Vendor Manager user, open an event owned by an Event Coordinator.	Event is visible, read only (Event_Sharing_For_Vendors).	Pending execution No result recorded.
TC-28	Assign the Feedback Manager permission set to an Event Coordinator user; edit and delete Feedback.	Read, create, edit and delete on Feedback are available.	Pending execution No result recorded.

11.2.4 Reporting, application and migration

Table 11.5: Test cases: reporting, application and migration

ID	Test scenario / input	Expected result	Status and actual result
TC-29	Import sample records for each object with the Data Import Wizard.	Records are imported without duplicates; counts match the source files.	Pending execution No result recorded.
TC-30	Run the report Upcoming Events by Month.	Events with date $\geq$ today and status $\neq$ Canceled are grouped by month with record count and budget sum.	Pending execution No result recorded.

ID	Test scenario / input	Expected result	Status and actual result
TC-31	Open the EventForce Operations Dashboard.	Dashboard renders the donut chart from the Upcoming Events by Month report.	Verified (evidence supplied) Dashboard screenshot shows the widget “Upcoming Events by Month.” with a record count of 1 and event date 10/5/2026, as of Oct 3, 2026, 7:13 AM. Only one record was displayed.
TC-32	Open the Event Planner app from the App Launcher.	App opens with tabs for all five objects, Reports and Dashboards.	Pending execution No result recorded.
TC-33	Run Apex unit tests.	Tests pass with at least 75% code coverage (needed for production deployment).	Not verified No Apex test classes are provided in the guide; cannot be executed.

11.3 Test Execution Summary

Table 11.6: Summary of planned and executed tests

Status	Number of test cases
Total planned test cases	33
Verified (evidence supplied)	3
Pending execution	28
Not verified	2

No pass percentage is reported, because the number of executed functional tests is not yet sufficient for one to be meaningful. The three verified items confirm the existence and state of the reminder Flow, the approval process and the dashboard; they do not confirm the run-time behaviour of the Flow or approval process. The pending test cases should be executed in the org and their results recorded in the last column before final submission, if the team wishes to claim them as performed.

11.4 Test Evidence Placeholders

The following placeholders are reserved for screenshots that evidence the key test cases once they are executed.

[INSERT SCREENSHOT HERE – Figure 11.1]

Capture: Client record edit page showing the error “Please Enter Valid Email Address” under the Email field.

Delete this box after inserting the image; keep the caption below.

Figure 11.1: Evidence for TC-04: invalid e-mail rejected

[INSERT SCREENSHOT HERE – Figure 11.2]

Capture: Event save attempt showing the error “This Venue is already booked on this date.”

Delete this box after inserting the image; keep the caption below.

Figure 11.2: Evidence for TC-09: double booking prevented

[INSERT SCREENSHOT HERE – Figure 11.3]

Capture: Venue record showing Availability Status = Reserved after the linked event is Confirmed.

Delete this box after inserting the image; keep the caption below.

Figure 11.3: Evidence for TC-11: venue status set to Reserved

[INSERT SCREENSHOT HERE – Figure 11.4]

Capture: Received reminder e-mail or e-mail log entry for the client; or the scheduled-path pending-actions view of the Flow.

Delete this box after inserting the image; keep the caption below.

Figure 11.4: Evidence for TC-14: reminder e-mail

[INSERT SCREENSHOT HERE – Figure 11.5]

Capture: Approval History related list on an Event showing the submission and the approved or rejected outcome with the resulting Event Status.

Delete this box after inserting the image; keep the caption below.

Figure 11.5: Evidence for TC-17 to TC-19: approval process execution

[INSERT SCREENSHOT HERE – Figure 11.6]

Capture: Apex Jobs page showing BatchCompleteEvents completed, and an Event whose status changed to Completed.

Delete this box after inserting the image; keep the caption below.

Figure 11.6: Evidence for TC-20: batch completion of past events

[INSERT SCREENSHOT HERE – Figure 11.7]

Capture: Screens obtained by logging in as an Event Coordinator, Vendor Manager and Client Profile user showing permitted and denied actions.

Delete this box after inserting the image; keep the caption below.

Figure 11.7: Evidence for TC-22 to TC-24: profile access

12. RESULTS AND DISCUSSION

12.1 Implementation Status

Table 12.1 compares what is documented in the implementation guide with the evidence that has been supplied so far.

Table 12.1: Implementation status by component

Component	Documented in guide	Evidence supplied	Status
Custom objects (6), tabs	Yes	None yet	Screenshot pending
Fields, relationships, formula, lookup filter	Yes	None yet	Screenshot pending
Validation rule Email_Valid_Address	Yes	None yet	Screenshot pending
Email templates (4)	Yes (text)	None yet	Screenshot pending
Approval process Event_Cancellation_Process	Yes	Setup screenshot: active on Event	Existence and activation confirmed; run-time not verified
Flow Client Reminder - 3 Days Before	Yes	Flows list screenshot: active	Existence and activation confirmed; run-time not verified
Apex classes and triggers	Yes (source code)	None yet	Screenshot pending
Batch and scheduled Apex job	Yes	None yet	Screenshot pending
Lightning App Event Planner	Yes	None yet	Screenshot pending
Report Upcoming Events by Month	Yes	Indirect: it is the source of the dashboard widget	Indirectly confirmed
EventForce Operations Dashboard	Yes	Dashboard screenshot	Confirmed; rendered with 1 record
Profiles, roles, users, permission set, OWD, sharing rule	Yes	None yet	Screenshot pending
Data migration (Data Import Wizard)	Stated only	None yet	Evidence not supplied
Production deployment	Described as out of scope	Not applicable	Not performed

12.2 Findings from the Supplied Evidence

- **Approval process.** The Approval Processes page for the Event object lists Event_Cancellation_Process as the only active process (process order 1) and shows no inactive processes. This confirms the process is configured and active; it does not show a cancellation request being submitted, approved or rejected.
- **Flow.** The Flows page lists “Client Reminder - 3 Days Before” as an active, unmanaged flow last modified by Preethi S on 10/3/2026. The label displays with a trailing comma in the list, which differs from the label recorded in the guide; this should be corrected or confirmed. The Flow’s execution has not been evidenced.
- **Dashboard.** The EventForce Operations Dashboard displays the Upcoming Events by Month widget as a donut chart with a record count of 1 for the event date 10/5/2026, as of Oct 3, 2026, 7:13 AM. This confirms the report-to-dashboard pipeline works. The display of a single record suggests that, at that time, only one event satisfied the report filters (date on or after today, status not Canceled).

12.3 Discussion

Table 12.2 relates each objective of Chapter 2 to the components built and to the current evidence.

Table 12.2: Objectives versus implementation and evidence

Objective	Component(s)	Evidence / status
Structured data model	Six custom objects, relationships, formula, lookup filter	Designed and documented; screenshots pending
Data accuracy	Validation rule; pre-validated import	Documented; test cases TC-04, TC-05, TC-29 pending
Venue availability and double-booking prevention	VenueStatusHelper, two triggers	Documented with source code; TC-09 to TC-12 pending; limitations identified (Chapter 13)
Automated reminders and status updates	Reminder Flow; batch and scheduled Apex	Flow active (evidence); batch and job pending
Controlled cancellation approval	Event_Cancellation_Process	Process active (evidence); TC-17 to TC-19 pending; two known issues
Role-based access	Profiles, roles, permission set, OWD, sharing rule	Documented; TC-22 to TC-28 pending; client restriction not documented
Operational visibility	Lightning App, report, dashboard	Dashboard rendered (evidence)
Documentation and maintenance readiness	This report; Chapter 10	Complete; production deployment not performed

The documentation shows that the design addresses every functional requirement of Chapter 3. The evidence supplied to date confirms the three components that are most visible in Setup and in the user interface, and the remaining components are documented in detail but await screenshots and test execution. Business-value targets (such as reducing manual effort by 60%) have not been measured and no performance improvement is claimed.

Several behaviours identified when reviewing the guide's source code and configuration, such as the treatment of cancelled events by the double-booking trigger and the nightly batch, affect the correctness of the automation under particular conditions. They are set out in Chapter 13 so that they can be verified in the org and corrected.

13. LIMITATIONS AND FUTURE ENHANCEMENTS

13.1 Advantages and Disadvantages

13.1.1 Advantages

- **Centralised data management** – event, client, vendor, venue, feedback and event–vendor data are managed in Salesforce;
- **Process automation** – a Flow automates reminders and an approval process controls cancellations;
- **Venue protection** – Apex logic validates venue availability and is designed to prevent booking conflicts;
- **Operational visibility** – a report and dashboard show upcoming events;
- **Controlled access** – profiles, roles, a permission set, OWD and a sharing rule limit data access;
- **Scalable platform** – the solution can grow with additional requirements.

13.1.2 Disadvantages

- **Developer Edition limitation** – the project was implemented for academic purposes in a standalone org;
- **Production deployment not performed** – migration to production was outside the scope;
- **Additional tooling needed** – real deployment needs sandboxes, change sets, Salesforce DX or DevOps tools;
- **Configuration complexity** – flows, Apex, approvals and security need Salesforce knowledge;
- **Ongoing maintenance** – automation and security settings need regular monitoring.

13.2 Items Identified in the Implementation Guide

Table 13.1 lists limitations and issues that can be identified from the guide’s configuration and source code. They are presented as **guide-identified limitations or items requiring verification**. Unless stated otherwise, the team’s evidence does not confirm whether any of them has occurred in the org or has been fixed; the status column therefore reads “Not verified in org”.

Table 13.1: Guide-identified limitations and items requiring verification

ID	Item identified in the guide	Potential impact	Suggested improvement	Status
L1	Rejection mismatch: the rejection action sets Event Status to Rejected, but the rejection e-mail text says the status “has been reverted to Confirmed”.	Event owners may be told the wrong status; confusion about the event’s real state.	Align the e-mail text with the status, or change the final-rejection field update if reversion to Confirmed is intended.	Not verified in org
L2	The e-mail template “Email to Client After Cancellation” is created but not attached to any approval action; only the event owner is notified on approval.	Clients are not informed automatically of an approved cancellation.	Attach a client e-mail through an alert or a Flow that uses the client’s e-mail address.	Not verified in org
L3	Double booking: PreventDoubleBooking compares only with events already saved and does not exclude Canceled or Rejected events; events inserted in the same batch are not compared with each other.	A cancelled event may still block a venue/date; duplicate bookings inside one bulk load may not be detected.	Exclude cancelled/rejected events from the query; compare records within Trigger.new; add unit tests.	Not verified in org
L4	Nightly batch: BatchCompleteEvents selects every event dated before today whose status is not Completed, which includes Canceled, Rejected and Pending Cancellation events.	Cancelled events may be marked Completed, distorting reports.	Restrict the query to statuses such as Planned and Confirmed, or as the business decides.	Not verified in org
L5	Event Budget is a fixed estimate by Event Type (formula), not an entered or calculated budget.	Budget does not reflect actual costs; all events of a type show the same value in reports.	Add an editable budget or roll up costs from EventVendor records.	Design limitation (by design)
L6	The Client Profile is meant to read “own events”, but no sharing mechanism restricting client users to their own events is documented.	Client users may see no events or too many, depending on ownership and sharing.	Define the mechanism, for example sharing rules, Apex managed sharing or an Experience Cloud site with sharing sets, and test it.	Not verified in org
L7	No Apex test classes are provided.	Apex cannot be deployed to production without sufficient coverage; regressions are not detected.	Write test classes for the triggers, helper, batch and schedulable classes.	Gap in guide

13.3 Additional Observations for Verification

The following further points were noted while reviewing the guide. They are also unverified and are listed so that the team can check them.

Table 13.2: Additional observations requiring verification

ID	Observation	Why it matters
O1	VenueStatusHelper adds one Venue__c record per event to a list before updating; if two events saved in one operation share a venue, the list may contain a duplicate Id.	Salesforce rejects an update with duplicate Ids; a Map keyed by venue Id avoids this.
O2	Venue status follows event status only. A Venue stays Reserved after its event is completed, and cancelling one event sets the Venue to Available even if another confirmed event uses it.	Venue availability may be inaccurate; availability by date would be more reliable.
O3	The Flow's entry condition checks only Event Status = Confirmed, while the scenario requires a client with an e-mail address; events confirmed less than three days before the event also need checking.	The e-mail action may fail or never fire in these cases.
O4	The guide's Flow body text shows "{!\$Record.Name}" with an extra brace.	May display incorrectly in the e-mail.
O5	The e-mail regular expression allows only letters, digits, dots and underscores before "@".	Valid addresses containing hyphens or plus signs would be rejected.
O6	Phase 1 scope names Flows for feedback notifications and approval updates, but the guide documents only the reminder Flow.	Scope and implementation are not fully aligned.
O7	Naming differs across the guide: app "Event Planner" versus "EventForce Management System application"; folder "EventForce Report" versus "EventForce Reports"; "Canceled" (Event) versus "Cancelled" (Vendor); flow label with a trailing comma in the org list.	Inconsistent names confuse users and reports.
O8	The Lightning App is assigned only to the System Administrator profile in the guide, while other profiles set Event Planner as default app.	Other users may not see the app until it is assigned to their profiles.
O9	The approval step uses a specific user as approver and no recall or escalation is described.	Requests can stall if the approver is unavailable.
O10	The dashboard displayed one record, whereas the guide recommends at least ten records per object.	More sample data is needed to demonstrate the report and dashboard.

13.4 Future Enhancements

- **Correct the identified issues** – resolve L1 to L7 and O1 to O10 after verification, and add Apex test classes;

- **Production deployment** – deploy from sandbox to production using change sets, Salesforce DX or a DevOps pipeline;
- **Advanced event automation** – additional reminders, feedback requests after events, and client notification of cancellations;
- **Enhanced vendor management** – vendor performance tracking and cost fields on EventVendor;
- **Advanced analytics** – more reports and dashboards on venues, vendors and client feedback;
- **External integrations and mobile access** – integrate with calendars, communication and payment services; improve mobile usability;
- **Client access** – a client portal with proper sharing to let clients view only their own events;
- **Continuous maintenance** – regular review of flows, Apex, validation rules and sharing settings.

14. CONCLUSION

The EventForce Management System demonstrates how Salesforce can be used to streamline and automate event planning operations. The design brings Events, Clients, Vendors, Venues, Feedback and Event–Vendor assignments into one platform and combines declarative tools (validation rule, formula field, lookup filter, Flow and Approval Process) with Apex triggers, a helper class, batch and scheduled Apex, a Lightning App, a report and a dashboard, secured by profiles, roles, a permission set, organisation-wide defaults and a sharing rule.

The supplied evidence confirms that the three-day reminder Flow and the cancellation approval process are active in the Developer Edition org and that the EventForce Operations Dashboard renders data from the Upcoming Events by Month report. The remaining components are documented in detail in the implementation guide and are marked in this report for screenshot insertion, and the test cases that have not yet been executed are shown as pending rather than passed.

The project also shows the importance of testing, security, documentation and maintenance in a Salesforce implementation. Review of the guide has identified limitations – for example the treatment of cancelled events by the double-booking trigger and the nightly batch, the rejection message mismatch and the absence of Apex test classes – that should be verified and corrected before any production use. Production deployment was not performed; the deployment-preparation, maintenance and documentation approach described in Chapter 10 provides the foundation for future migration, scalability and enhancement.

REFERENCES

- [1] EventForce Management System – Salesforce Implementation, project implementation guide (project documentation for this project; includes user story, ideation, requirement analysis, design, planning and phase-wise milestones).
- [2] Salesforce Developer Project Guide: Project Completion Guide, EventForce Management System track, Final Project Submission, 3 October 2026.
- [3] Salesforce, “Salesforce Help,” <https://help.salesforce.com> (documentation on objects, fields, flows, approval processes, reports, dashboards, profiles, permission sets and sharing).
- [4] Salesforce, “Apex Developer Guide,” <https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/> (triggers, classes, Batch Apex, Schedulable Apex, testing and code coverage).
- [5] Salesforce, “Developer Edition sign-up,” <https://developer.salesforce.com/signup>.
- [6] Salesforce, “Salesforce DX Developer Guide,” https://developer.salesforce.com/docs/atlas.en-us.sfdx_dev.meta/sfdx_dev/ (source-driven development and deployment).
- [7] Salesforce Trailhead, <https://trailhead.salesforce.com> (learning modules on Salesforce administration and development).

APPENDIX A: SOURCE CODE AND EXPRESSIONS

The code below is reproduced from the implementation guide [1] for documentation purposes. Its behaviour in the org is subject to the test cases and the observations in Chapters 11 and 13.

A.1 Event Budget Formula (Event__c)

```
CASE(
  TEXT(Event_Type__c),
  "Wedding", 50000,
  "Corporate", 30000,
  "Birthday", 10000,
  "Anniversary", 20000,
  "Festival", 60000,
  "Concert", 40000,
  "Other", 15000,
  0
)
```

A.2 Validation Rule Email_Valid_Address (Client__c)

```
NOT(Regex(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9._]+\.[a-zA-Z]{2,}$"))
```

Error message: “Please Enter Valid Email Address”; location: Email field.

A.3 VenueStatusHelper (Apex class)

```
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c,
                    Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c,
                    Availability_Status__c='Available'));
                }
            }
        }
        if(!venues.isEmpty()) update venues;
    }
}
```

A.4 EventTrigger13 (Apex trigger)

```
trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Trigger.new);
}
```

A.5 PreventDoubleBooking (Apex trigger)

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
                      FROM Event__c WHERE Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
                    ev.addError('This Venue is already booked on this date.');
```

A.6 BatchCompleteEvents (Batch Apex)

```
global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc){
        return Database.getQueryLocator(
            'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c WHERE ' +
            'Event_Date__c < TODAY AND Event_Status__c != \'Completed\''
        );
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope){
        for(Event__c ev : scope){
            ev.Event_Status__c = 'Completed';
        }
        update scope;
    }
    global void finish(Database.BatchableContext bc){
        System.debug('Past events marked as Completed');
```

In the guide the query string is written across two lines; it is shown here with a concatenation operator for readability.

A.7 ScheduleCompleteEvents (Schedulable Apex)

```
global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}
```

APPENDIX B: EMAIL TEMPLATES

The four classic text e-mail templates below are recorded in the implementation guide [1].

B.1 Event Cancellation Request Notification

Subject: Approval Request: Cancel Event {!Event__c.Name}

```
Hello!
An approval request has been submitted to cancel the event:
- Event Name: {!Event__c.Name}
- Event Date: {!Event__c.Event_Date__c}
Thank you,
EventForce Management System
```

B.2 Email to Client After Cancellation (not attached to an action – see L2)

Subject: Event Cancellation Notice – {!Event__c.Name}

```
Hello!
We would like to inform you that the event has been canceled.
Event Name: {!Event__c.Name}
Date: {!Event__c.Event_Date__c}
If you have any questions or need assistance, please contact us.
Thank you for your understanding,
Event Management Team
```

B.3 Approval Notification to Event Owner

Subject: Event Cancellation Approved: {!Event__c.Name}

```
Hello {!Event__c.OwnerFullName}
Your cancellation request for the event {!Event__c.Name} has been approved.
Event Date: {!Event__c.Event_Date__c}
The event status has been updated to Canceled.
Thank you,
EventForce Management System
```

B.4 Rejection Notification to Event Owner (see L1)

Subject: Event Cancellation Rejected: {!Event__c.Name}

```
Hello {!Event__c.OwnerFullName},
Your cancellation request for the event {!Event__c.Name} has been rejected.
Event Date: {!Event__c.Event_Date__c}
The event status has been reverted to Confirmed.
Thank you,
EventForce Management System
```

B.5 Flow Text Templates

```
Subject: Reminder: Your Event {!$Record.Name} is in 3 days
```

```
Body:
Hello {!$Record.Client__r.Name},
This is a reminder that your event "{!$Record.Name}" will take place on
{!$Record.Event_Date__c}.
Venue: {!$Record.Venue__r.Name}
Type: {!$Record.Event_Type__c}
We look forward to seeing you!
- EventForce Team
```

The body line containing “{!\$Record.Name}” is reproduced exactly as in the guide and is flagged in Chapter 13 (O4).

APPENDIX C: SCREENSHOT PLACEHOLDER REGISTER

This register lists every labelled screenshot placeholder in the report so that the screenshots can be inserted in order. After inserting an image, delete the grey placeholder box and keep the caption. Table C.1 also shows where the team has already supplied a screenshot.

Table C.1: Screenshot placeholders to be completed

Figure	Screen or feature to capture	Supplied?
7.1	Salesforce Setup home page of the Developer Edition org	No
7.2	Custom objects in Object Manager	No
7.3	Custom object tabs	No
7.4	Fields of the Event object	No
7.5	Fields of the Client object	No
7.6	Fields of the Vendor and Venue objects	No
7.7	Fields of the Feedback object	No
7.8	EventVendor junction object relationships	No
7.9	Event Budget formula field	No
7.10	Lookup filter on the Feedback Event field	No
7.11	Validation rule Email_Valid_Address	No
7.12	Classic email templates	No
7.13	Event_Cancellation_Process in Approval Processes	Yes – crop and insert
7.14	Approval process detail page with actions	No
7.15	Client Reminder flow in the Flows list	Yes – crop and insert
7.16	Flow canvas of the 3-day reminder Flow	No
7.17	Apex classes in Setup	No
7.18	Apex triggers in Setup	No
7.19	Apex source code in the Developer Console	No
7.20	Scheduled job Daily Event Completion	No
8.1	Event Planner Lightning App	No
8.2	App Manager entry and app settings	No
8.3	Report folder and sharing	No
8.4	Upcoming Events by Month report	No
8.5	EventForce Operations Dashboard	Yes – crop and insert

Figure	Screen or feature to capture	Supplied?
9.1	Data Import Wizard	No
9.2	Sample records in EventForce objects	No
9.3	Custom profiles	No
9.4	Object permissions of a custom profile	No
9.5	Role hierarchy	No
9.6	Users with roles and profiles	No
9.7	Feedback Manager permission set and assignment	No
9.8	Organisation-wide defaults for Event	No
9.9	Sharing rule Event_Sharing_For_Vendors	No
11.1	Evidence for TC-04: invalid e-mail rejected	No
11.2	Evidence for TC-09: double booking prevented	No
11.3	Evidence for TC-11: venue status set to Reserved	No
11.4	Evidence for TC-14: reminder e-mail	No
11.5	Evidence for TC-17 to TC-19: approval process execution	No
11.6	Evidence for TC-20: batch completion of past events	No
11.7	Evidence for TC-22 to TC-24: profile access	No

APPENDIX D: IMPLEMENTATION EVIDENCE SUPPLIED BY THE TEAM

The following screenshots, supplied by the team, were used to establish what can be stated as confirmed. They are described here but not reproduced; the team may insert them in the placeholders listed in Appendix C.

Table D.1: Description of supplied evidence

Evidence	What it shows	Used for
EventForce Operations Dashboard (Lightning)	Dashboard “EventForce Operations Dashboard”, viewed as Preethi S, as of Oct 3, 2026, 7:13 AM. Widget “Upcoming Events by Month.” as a donut chart; record count 1; event date 10/5/2026; link “View Report (Upcoming Events by Month)”.	TC-31; Chapter 8; Chapter 12
Setup → Approval Processes	Manage Approval Processes For: Event. Active Approval Processes: Event_Cancellation_Process, process order 1. Inactive Approval Processes: none.	TC-16; Chapters 7 and 12
Setup → Flows (All Flows)	Row “Client Reminder - 3 Days Before,” process type Autolaunched Flow, Active ticked, not a template, Unmanaged, last modified by Preethi S on 10/3/2026, 3:06 AM.	TC-13; Chapters 7 and 12

The supplied screenshots also display browser tabs, the organisation’s URL and desktop details. These should be cropped out before the images are inserted in the report.